NOVA SCHOOL OF SCIENCE & TECHNOLOGY

Computer Networks

Redes de Computadores

RC

Teórica DEZ

LEI

João Leitão

©2026 NOVA FCT

A Última Aula

- **Endereçamento:** o endereçamento por classes e porque se esgotou, CIDR e máscaras de rede, hierarquia, e os intervalos especiais/reservados.

- **A esgotar os endereços:** o NAT permite que toda uma rede privada partilhe um único endereço público; o IPv6 contorna a escassez por completo com um espaço de 128 bits.

- **Multicast:** entrega de um para muitos a um grupo, em vez de repetir o mesmo unicast muitas vezes.

Modelo OSI


Camada de Aplicação
Camada de Apresentação
Camada de Sessão
Camada de Transporte
Camada de Rede
Camada de Ligação de Dados
Camada Física

O Que Cobrimos Até Agora



Camada de Transporte:

foi aí que estivémos durante quase um mês — UDP, fluxo de bytes ordenado e fiável no TCP, e controlo de saturação.

O Que Cobrimos Até Agora



Camada de Rede:

acabámos de a recapitular: endereçamento (IPv4), NAT e IPv6, multicast. O encaminhamento e o routing, o resto desta camada, ainda estão para vir.

Duas Camadas Nunca Abertas



Física (L1):

"transferir bits em bruto entre dispositivos através de um meio" — a definição que lhe demos no primeiro dia. O que nos interessa, já sabemos: move bits numa ligação, a uma certa taxa, com algum atraso, e de vez em quando com erros. Como é que os bits se tornam tensões elétricas, luz ou ondas de rádio é território da engenharia eletrotécnica — para nós, é um serviço sobre o qual construímos.

Duas Camadas Nunca Abertas



Ligação de Dados (L2):

“tratar da comunicação entre dois dispositivos diretamente ligados” — a mesma definição do primeiro dia. Já referimos esta camada várias vezes, mas esta nunca foi totalmente explicada — até hoje.

Porque as Saltámos?



A verdadeira razão para isto:

os projetos precisavam primeiro de sockets e de um protocolo de transporte, não destas duas camada. Hoje, abrimos a Ligação de Dados. A Física fica fechada de propósito: o que precisamos dela, já temos. Esta aula: como é que os switches decidem para onde enviar uma trama. Próxima aula: o que viaja realmente dentro de uma trama Ethernet, e o que é um endereço MAC. A aula seguinte: como é que um host entra na rede, e a começa a usar — DHCP, ARP, e o gateway.

Aula Dez

- **Auto-aprendizagem:** como um switch constrói a sua própria tabela de encaminhamento a partir do zero, sem qualquer configuração.

- **O problema dos ciclos, e a sua solução:** o que corre mal quando uma rede tem mais do que um caminho entre dois switches, e o protocolo que o resolve sem abdicar extritamente da redundância.

- **A comutação, generalizada:** três formas diferentes de uma rede fazer passar dados por várias ligações — comutação por pacote, por circuitos virtuais, e com encaminhamento na origem.

Uma Pergunta do Primeiro Dia

- **No primeiro dia, perguntámos:** “Como é que todos estes switches e routers decidem para onde enviar estas mensagens? São todos iguais?”

- **Também no primeiro dia, o mapa da cadeira indicava:** “Comutação e Ethernet” como um tema que iriamos discutir.

- **Hoje, começa a metade dessa resposta que diz respeito aos switches.**

O Que Já Sabem Sobre Switches

- **Store-and-forward:** um switch recebe a trama completa antes de a encaminhar — viram isto, salto a salto, há várias teóricas atrás.

- **Destino conhecido:** procura o endereço de destino e encaminha por uma única interface (mas operam na Camada 2; os endereços IP são da Camada 3 do Modelo OSI).

- **Destino desconhecido:** inunda a trama por todas as interfaces exceto a de chegada.
- **A questão em aberto:** ninguém configura essa tabela à mão. De onde vem ela, afinal?

Switch ou Bridge?

Rede (L3)	**Router**	**Lê:** o endereço IP de cada pacote. **Faz:** junta LANs separadas numa inter-rede.
Ligação de Dados (L2)	**Bridge = Switch**	**Lê:** o endereço MAC de destino de cada trama — recebe todas as tramas em todas as portas. **Faz:** encaminha-a pela porta certa, ou inunda. A aula de hoje.
Física (L1)	**Repetidor / Hub**	**Lê:** nada — apenas bits. **Faz:** copia-os cegamente para todas as outras portas. Sem endereços, sem tabela.

"Bridge" é o nome histórico de uma caixa que une duas LANs; um switch é uma bridge com muitas portas (ou muitas bridges colocadas no mesmo dispositivo). O mesmo dispositivo, o mesmo trabalho.

De Onde Vêm as Tabelas dos Switches?

Pergunta:
um switch procura um endereço de destino numa tabela para decidir para onde encaminhar. Ninguém escreve essa tabela à mão. Então como é essa tabela criada, e como é que o switch começa a saber alguma coisa?

Aprendizagem: Tabela Vazia

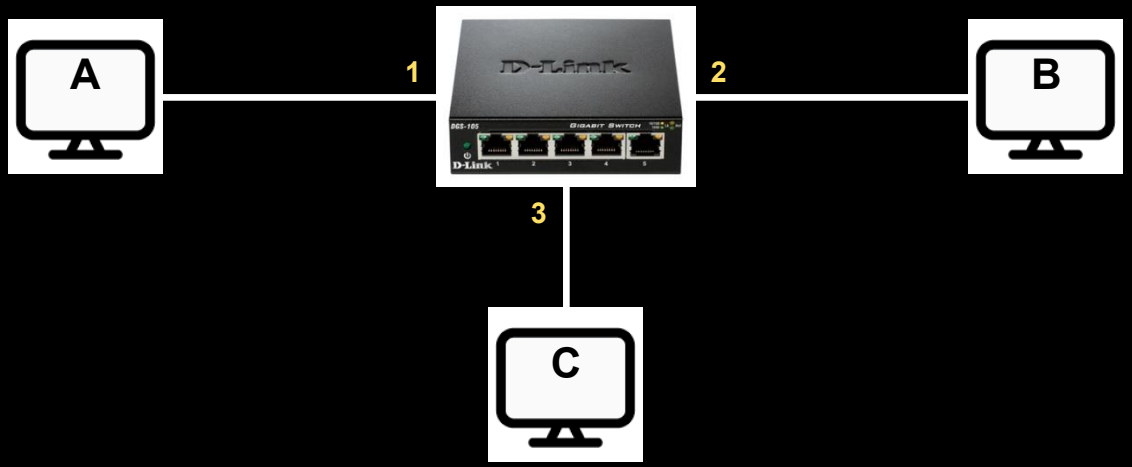


Tabela de encaminhamento

Porta	Endereço
1	—
2	—
3	—

Um switch sem configuração começa sem saber nada sobre quem (host) está onde. As suas portas estão numeradas 1, 2 e 3.

Aprendizagem: A Envia para C

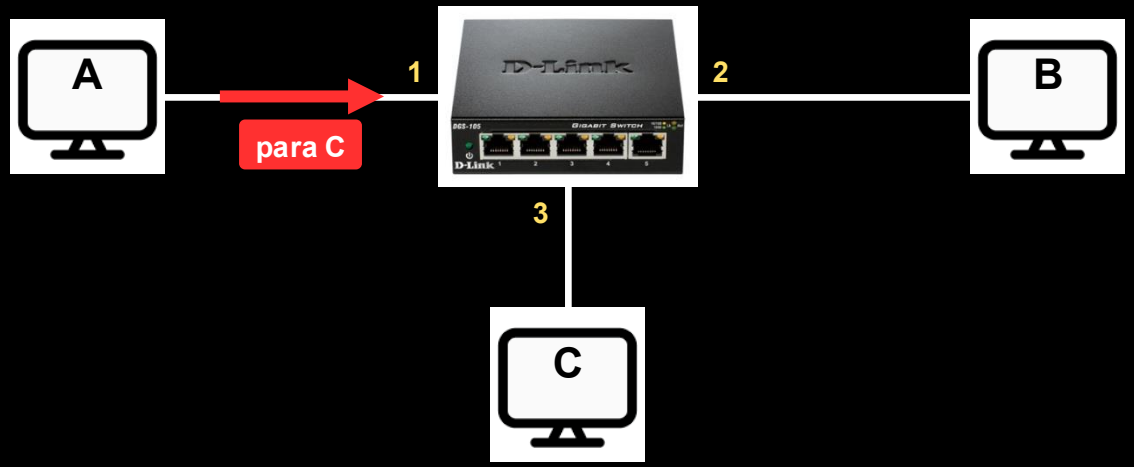


Tabela de encaminhamento

Porta	Endereço
1	—
2	—
3	—

O Host A envia uma trama dirigida ao Host C. Ela chega pela porta 1. O switch nunca viu uma trama antes.

Aprendizagem: Regista A

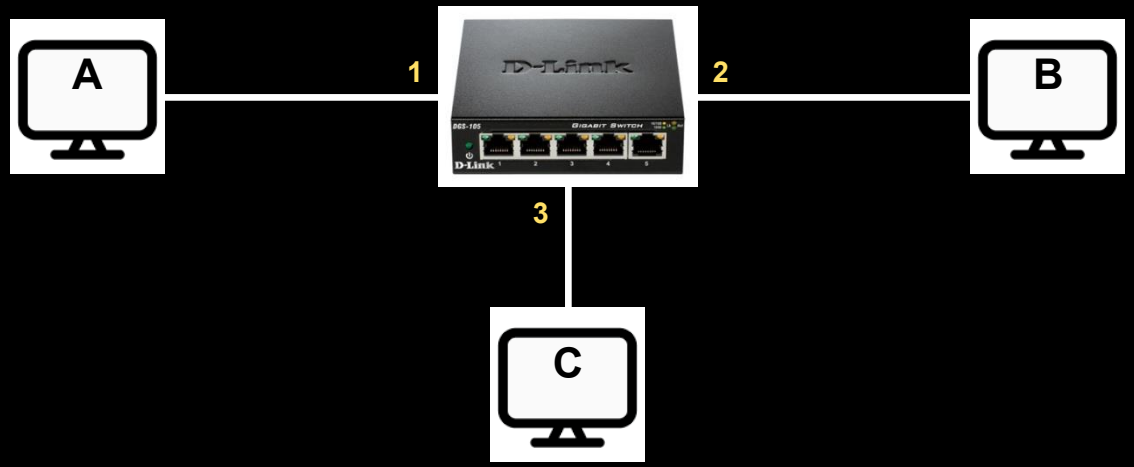


Tabela de encaminhamento

Porta	Endereço
1	A
2	—
3	—

Seja qual for a porta por onde uma trama chega, essa porta leva de volta à sua origem: a porta 1 corresponde agora a A.

C Ainda É Desconhecido: Inundar

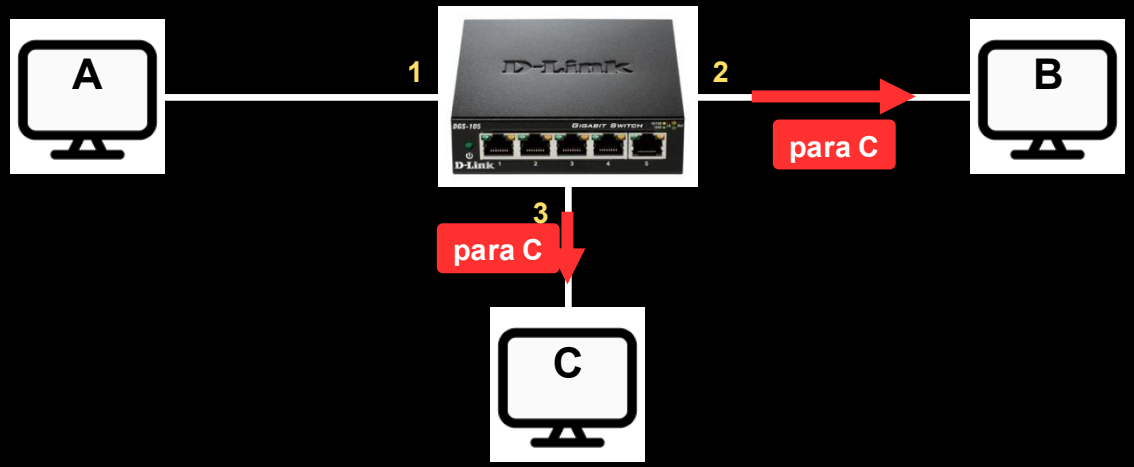


Tabela de encaminhamento

Porta	Endereço
1	A
2	—
3	—

O endereço de C ainda não está na tabela, por isso o switch faz a única coisa segura: inunda, por todas as portas exceto aquela por onde a trama chegou.

Aprendizagem: Só C Aceita

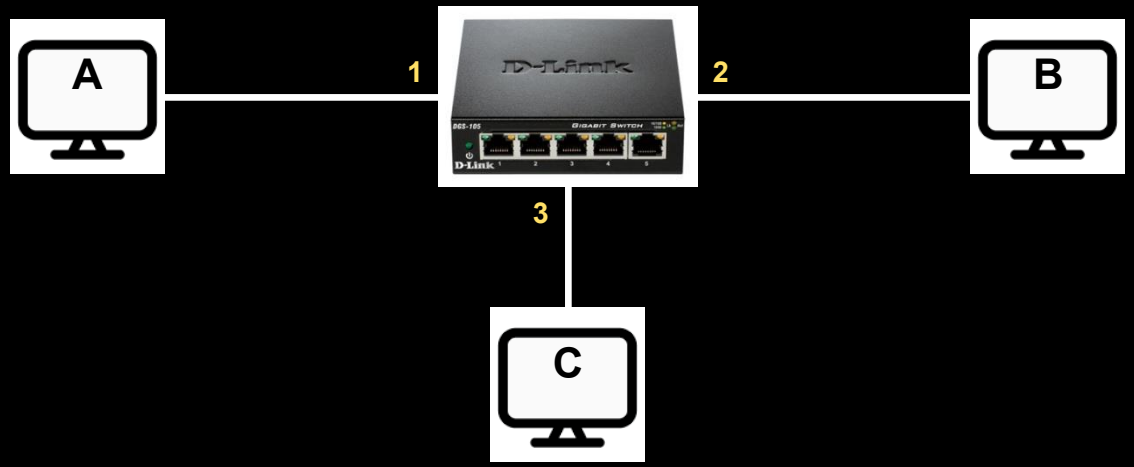


Tabela de encaminhamento

Porta	Endereço
1	A
2	—
3	—

O Host B vê a trama e ignora-a — não é dirigida a B. O Host C vê o seu próprio endereço e aceita-a.

Aprendizagem: C Responde a A

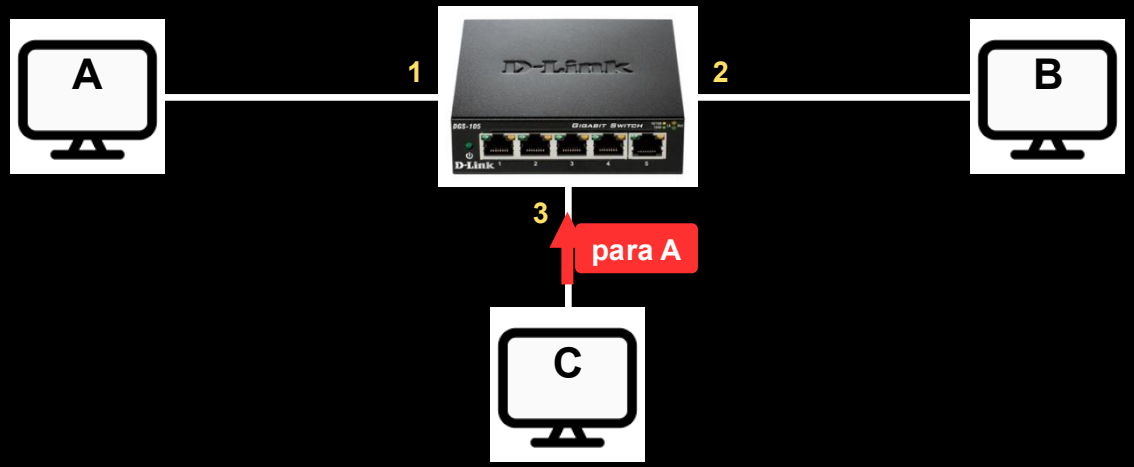


Tabela de encaminhamento

Porta	Endereço
1	A
2	—
3	—

O Host C responde, para o Host A. A resposta chega pela porta 3.

Aprendizagem: Regista C

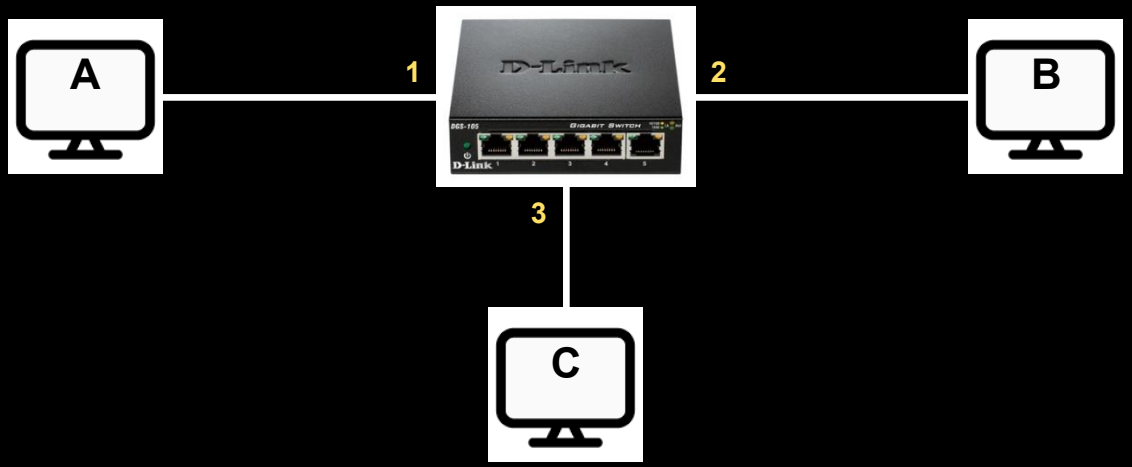


Tabela de encaminhamento

Porta	Endereço
1	A
2	—
3	**C**

A mesma regra, no mesmo instante: esta trama entrou pela porta 3, por isso a porta 3 corresponde agora a C.

Encaminhar em Vez de Inundar

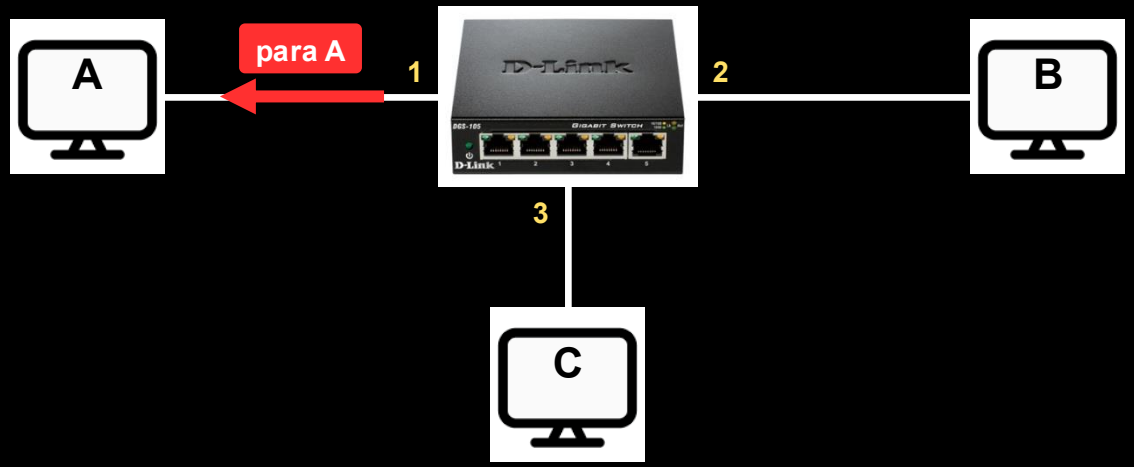


Tabela de encaminhamento

Porta	Endereço
1	A
2	—
3	C

A está na tabela. A resposta de C sai por exatamente uma porta — a porta 1. O Host B nunca a vê.

Auto-aprendizagem

Conclusão:
um switch constrói a sua própria tabela de encaminhamento observando o endereço de origem de cada trama que vê — sem configuração, sem troca de mensagens, apenas “a porta por onde isto chegou leva de volta ao seu emissor.” É exatamente o que uma aula anterior referiu, de passagem, como “um switch aprende onde vive cada endereço unicast” — agora conhecem o mecanismo.
(As entradas também expiram — um host que muda de porta é simplesmente reaprendido.)

A Tabela por Dentro

```
#define TABLE_SIZE 1024  /* máx. entradas    */
#define MAX_TTL     120  /* segundos de vida */

void updateTable(MacAddr src, int inif) {
    Entry *e = lookup(src);
    if (e == NULL) {           /* endereço novo */
        if (numEntries == TABLE_SIZE)
            return;            /* cheia: ignora */
        e = insert(src);
        numEntries++;
    }
    e->TTL = MAX_TTL;          /* renova        */
    e->ifnumber = inif;        /* última porta  */
}
```

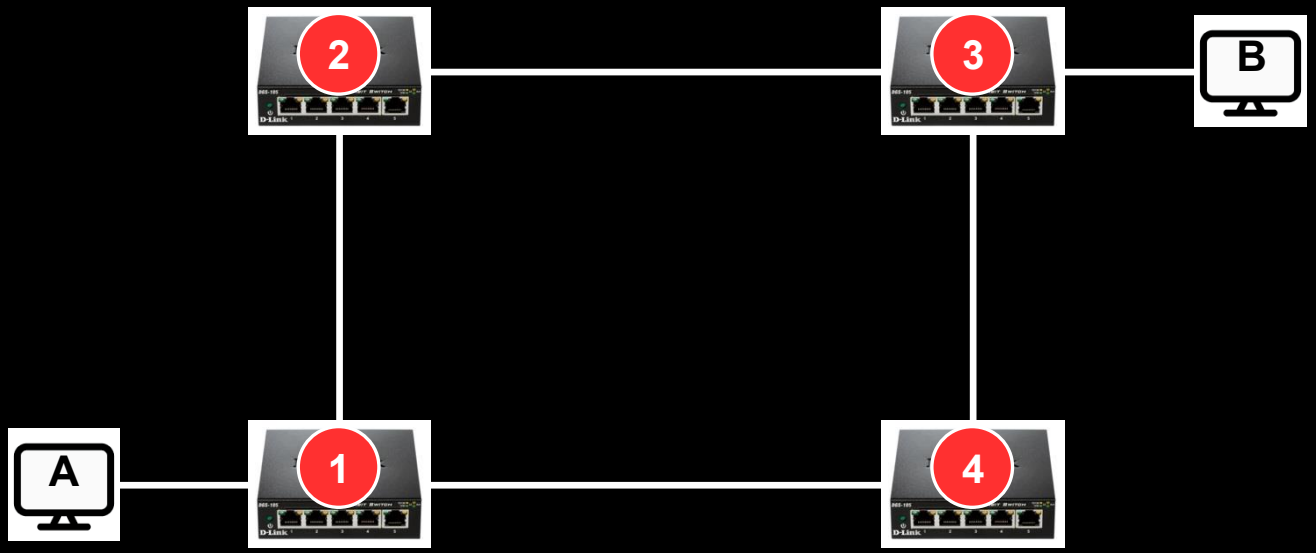
As entradas expiram: um host que muda de sítio ou fica calado é esquecido e reaprendido.

Tabela cheia? nada falha: os destinos desconhecidos são inundados. A tabela é uma otimização, não uma condição de correção.

O reverso: inundem um switch com origens falsas e a tabela enche — os hosts reais deixam de ser aprendidos e as suas tramas são inundadas para todos, incluindo quem escuta.

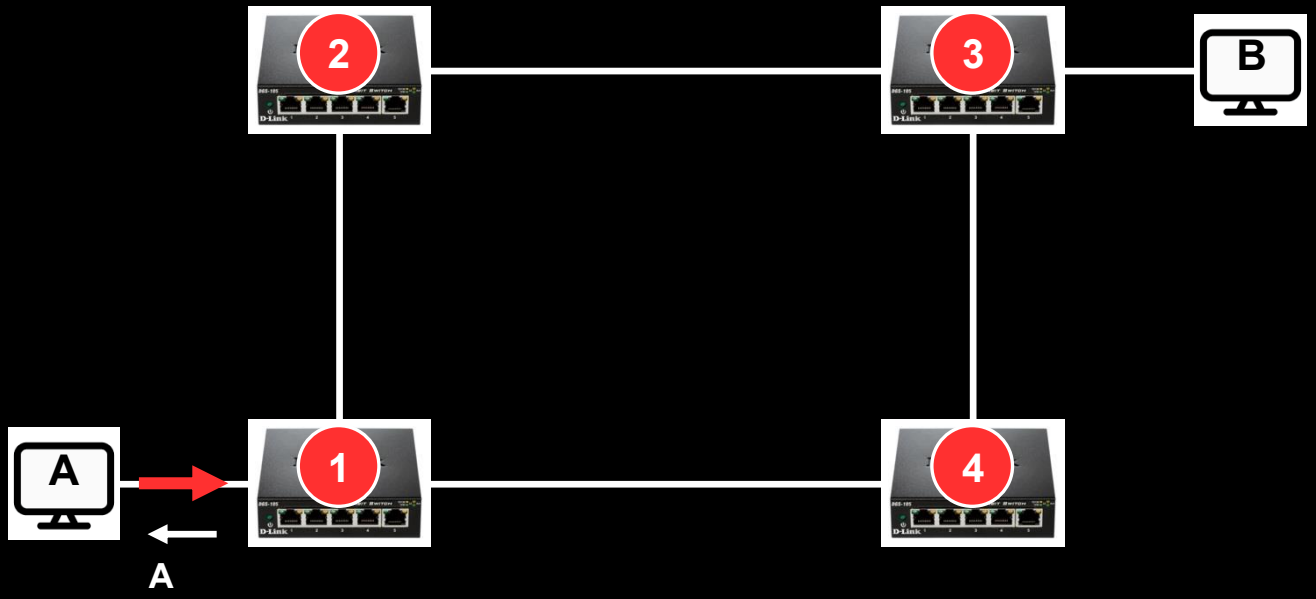
O passo de aprendizagem que cada switch corre para cada trama que chega. Uma tarefa separada, com um temporizador, decrementa cada TTL e remove as entradas que chegam a 0.

Mais do Que um Caminho?



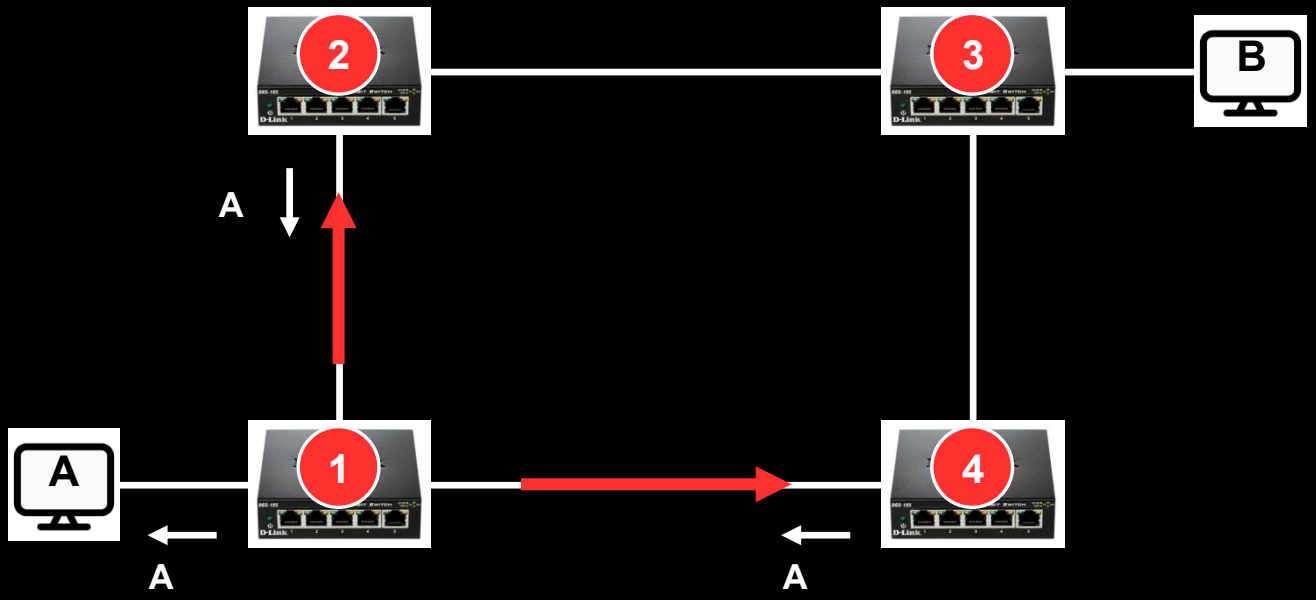
Pergunta:
Quatro switches ligados em quadrado — cada par de vizinhos tem duas formas de chegar um ao outro, de propósito, por fiabilidade.
As tabelas estão vazias.
O Host A envia uma trama para o Host B.
O que acontece a essa trama?

A Envia uma Trama para B



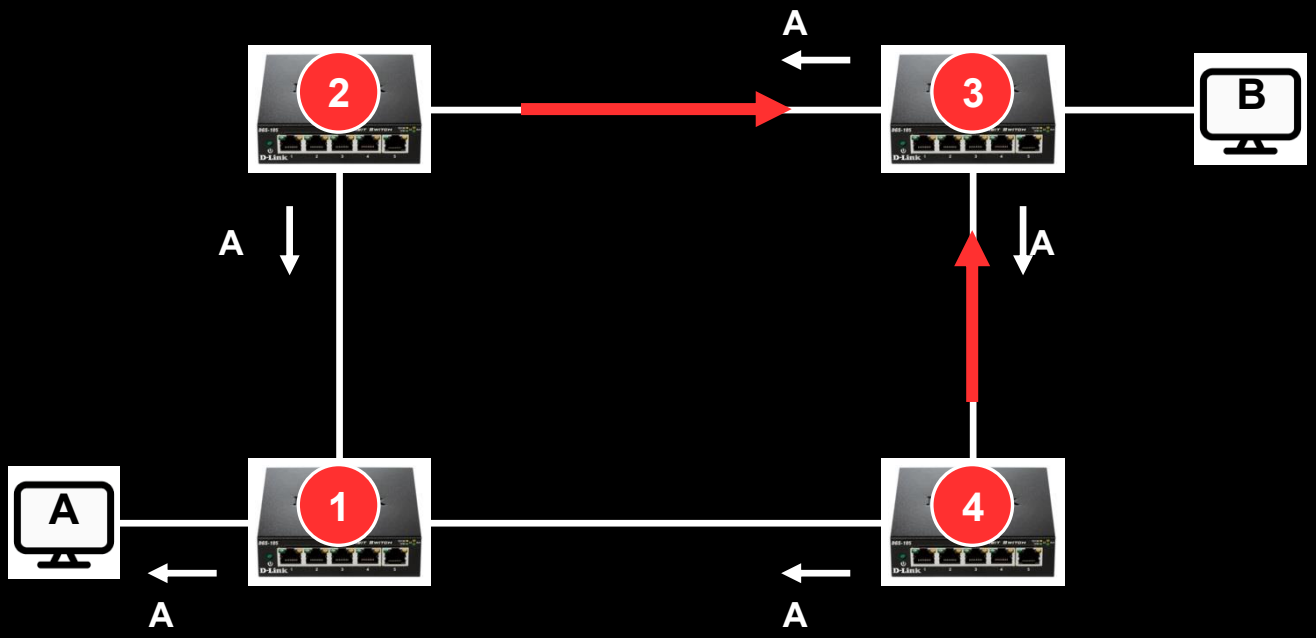
Todas as tabelas estão vazias. A trama de A chega ao switch 1, que aprende que A está do lado da porta virada para A.

O Switch 1 Inunda-a



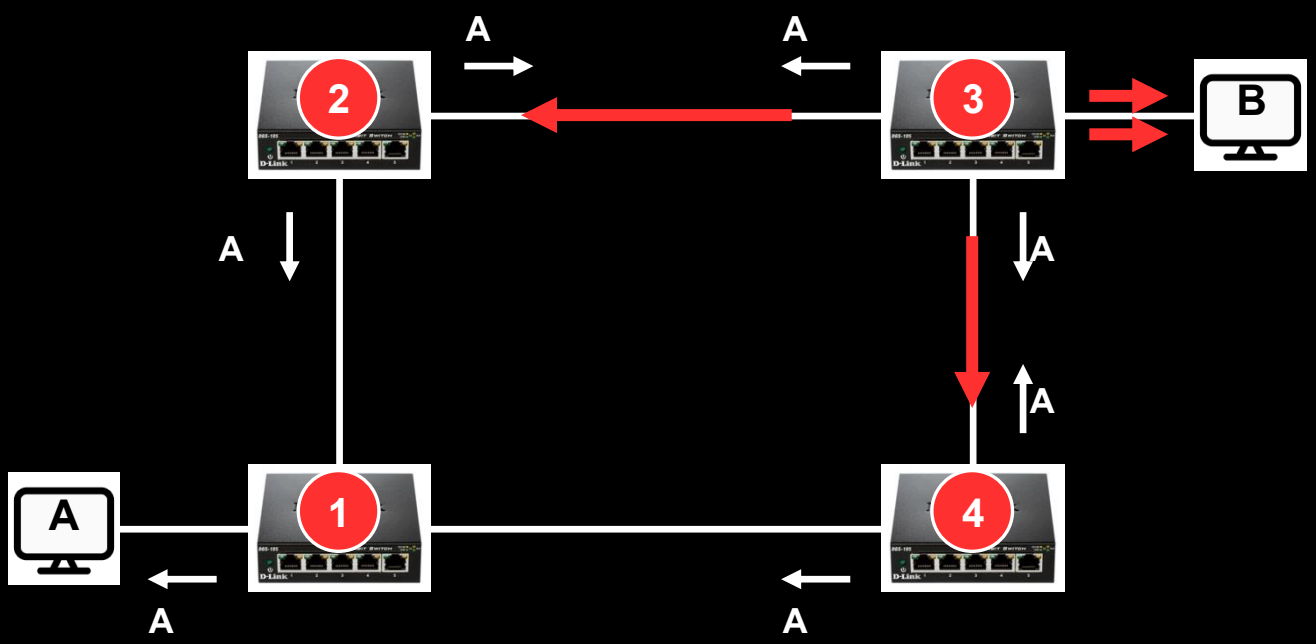
B é desconhecido, por isso o switch 1 inunda pelas outras duas portas. Os switches 2 e 4 recebem cada um uma cópia e aprendem: A está atrás do switch 1.

Duas Cópias Chegam ao Switch 3



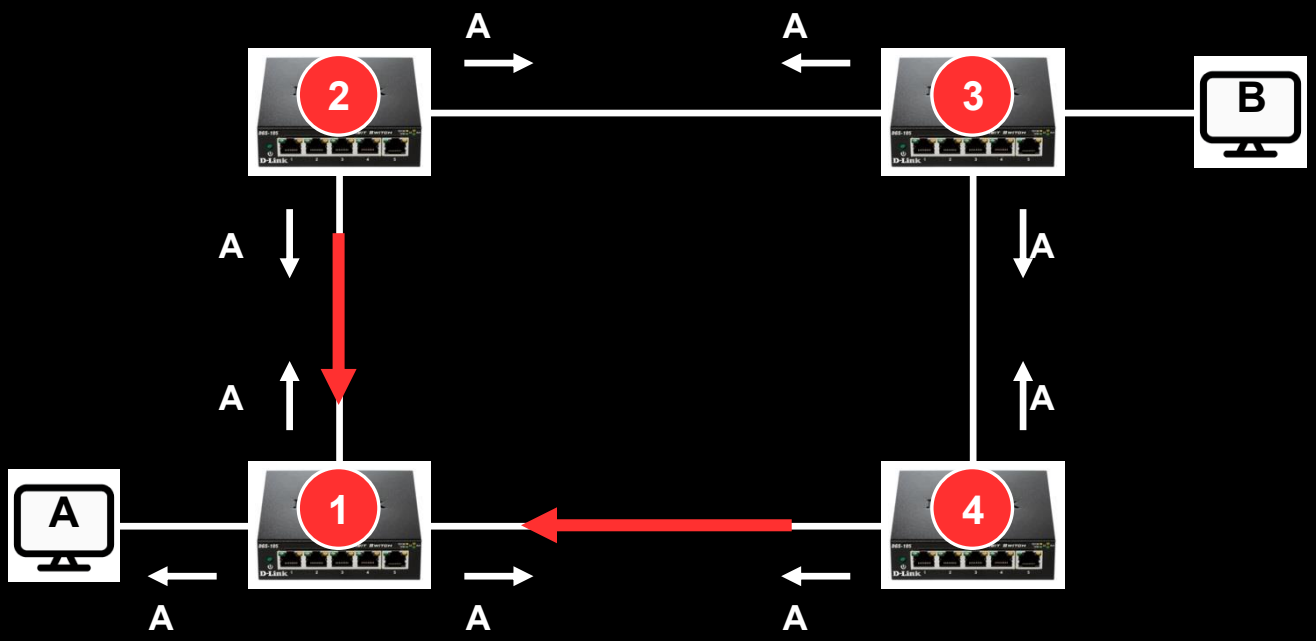
Os switches 2 e 4 também inundam. O switch 3 recebe uma cópia vinda do 2 e outra vinda do 4 — então para que lado está A? A sua tabela muda para a cópia que tiver chegado por último.

B Recebe-a — Duas Vezes



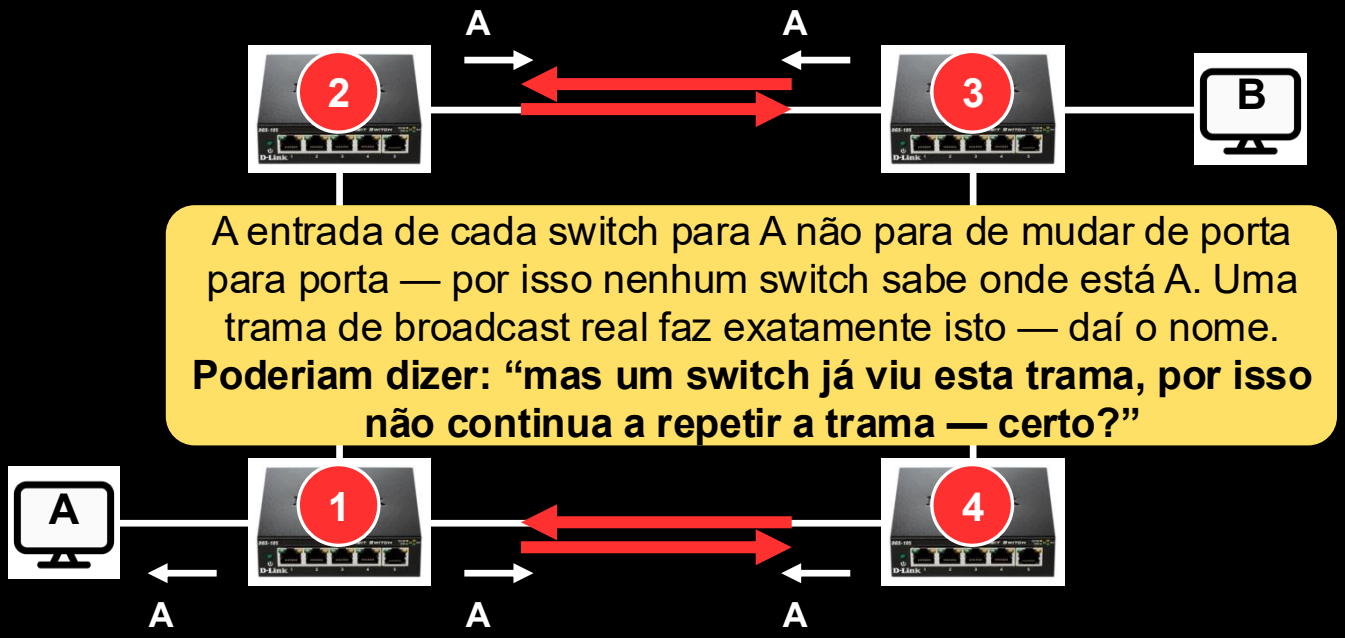
O switch 3 inunda cada cópia: B recebe a mesma trama duas vezes, e novas cópias seguem de volta para o 2 e o 4. Agora o 2 e o 4 pensam que A está atrás do switch 3.

De Volta ao Switch 1



As cópias voltam ao switch 1 — vindas do 2 e do 4. O switch 1 passa a acreditar que A também está lá em cima, e volta a inundá-las. Nada para isto.

Uma Tempestade de Broadcast



A Trama Sabe?



Pergunta: há aqui alguma coisa que identifique esta trama de forma única — algo que um switch pudesse usar para reconhecer uma cópia que já encaminhou?

Um olhar rápido para dentro da trama que cada switch encaminha — vamos ver todos os detalhes na próxima aula.

Não: Nada Marca uma Cópia



Sem número de sequência, sem ID, sem contador de saltos a diminuir em cada switch — o TTL do IP existe precisamente por causa dos ciclos. Mesma Origem, Destino e Dados — a cópia é igual ao original.

Duas cópias da mesma trama são idênticas bit a bit. Um switch não tem forma de as distinguir, por isso inunda todas as cópias, para sempre.

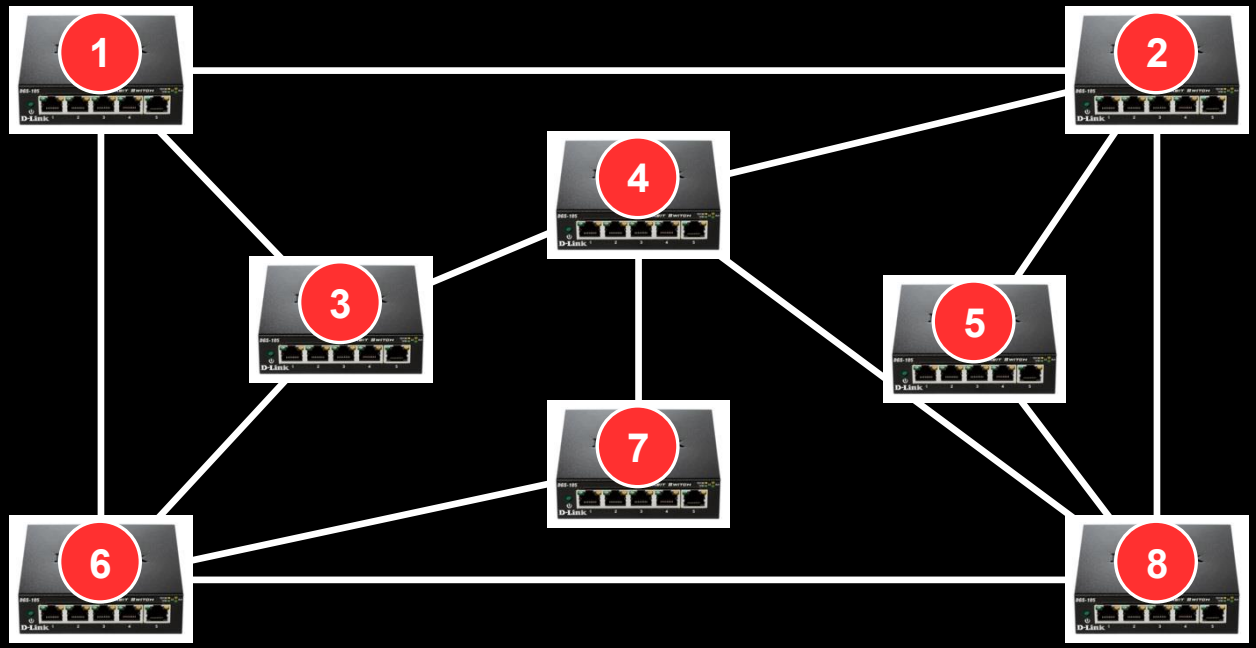
O Problema dos Ciclos

Conclusão:
a inundação e as ligações redundantes são, cada uma por si, inofensivas. Juntas, sem forma de dizer “já encaminhei esta trama antes”, combinam-se numa tempestade de broadcast infinita. A auto-aprendizagem nunca causou isto — o problema esteve sempre lá.

Manter as Ligações, Sem Ciclos

Pergunta:
não queremos remover as ligações redundantes — estão lá por fiabilidade. Será que os switches conseguem coordenar entre si de forma a deixar de encaminhar em apenas as ligações suficientes para quebrar todos os ciclos, mantendo essas ligações ligadas como reserva?

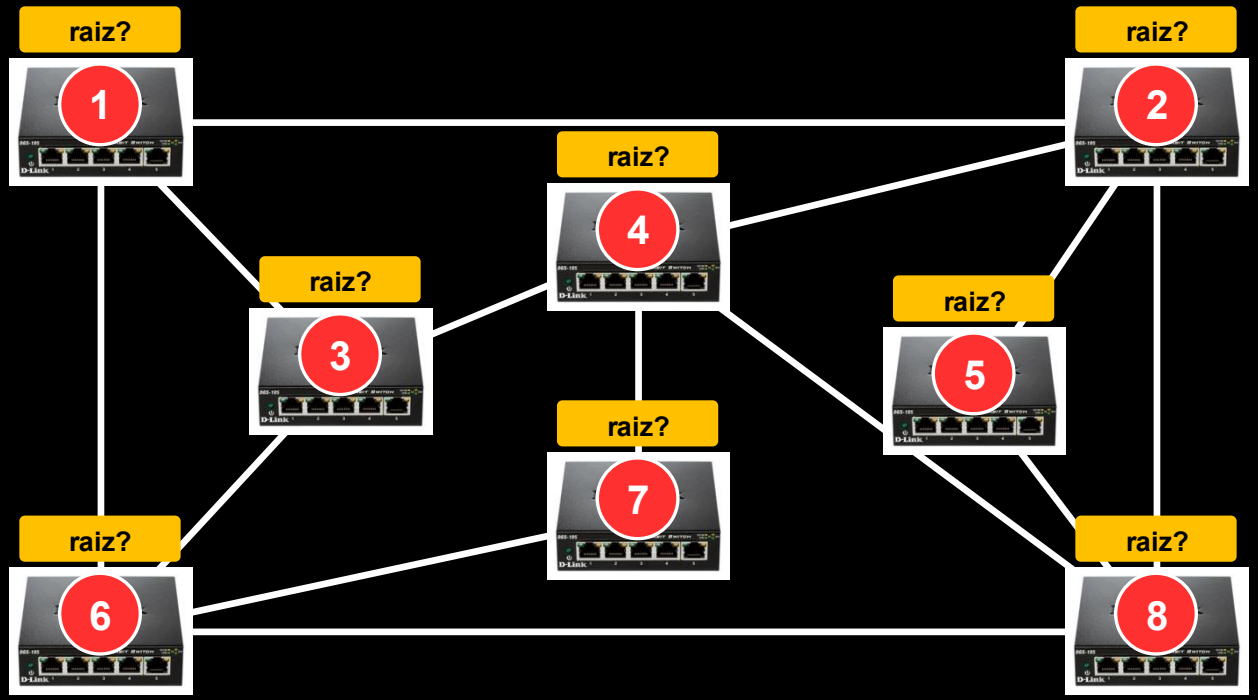
Uma Rede com Vários Ciclos



O mesmo objetivo de antes:
manter todas as ligações ligadas, mas encaminhar só nas necessárias para chegar a cada switch uma única vez — uma árvore de cobertura (spanning tree).

Oito switches, treze ligações, e ciclos por todo o lado — 1–2–4–3, 1–3–6, 2–5–8, 6–7–4–8… Ciclos como estes aparecem de propósito (redundância) ou por acidente, quando vários administradores acrescentam ligações sem ver a imagem completa. Cada switch tem um identificador único; aqui, simplesmente o seu número.

Todos Começam como Raiz

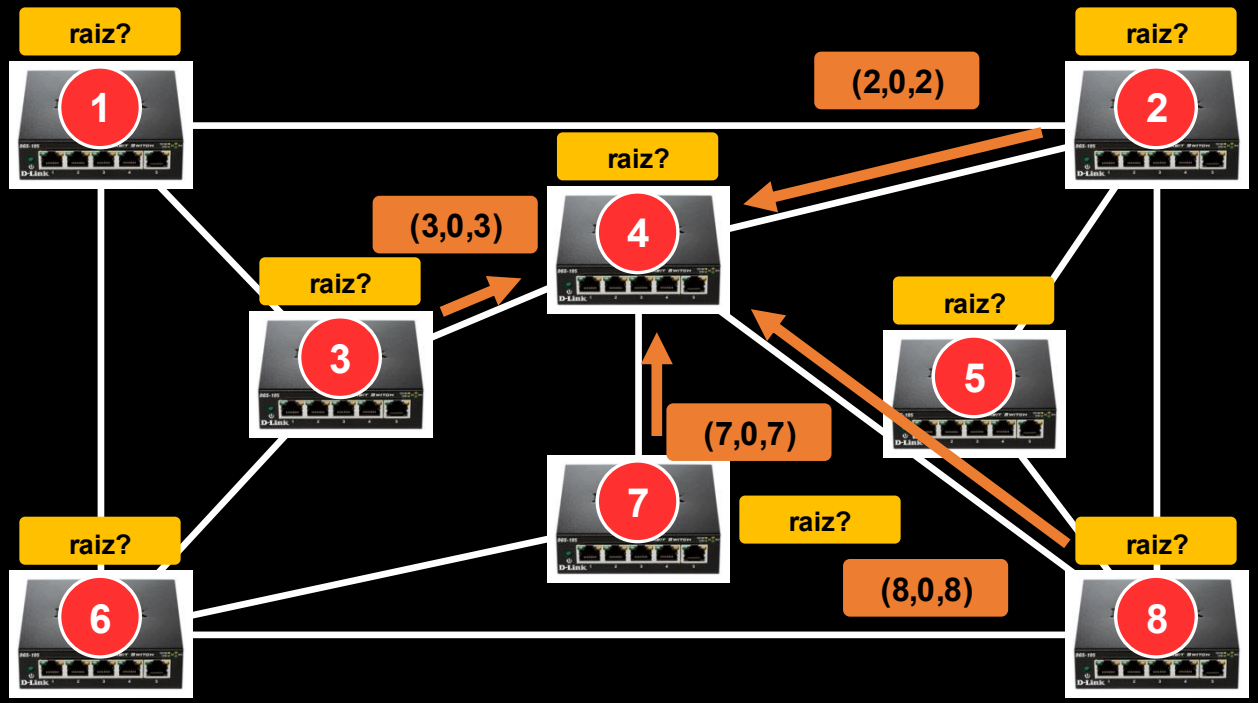


Mensagem de configuração:
(ID da raiz, distância à raiz, ID do emissor)

O switch 4 começa por enviar (4, 0, 4).

Ao ligar, nenhum switch sabe nada. Cada um anuncia (o seu próprio ID, distância 0, o seu próprio ID) em todas as portas.

O Que o Switch 4 Ouve ao Ligar

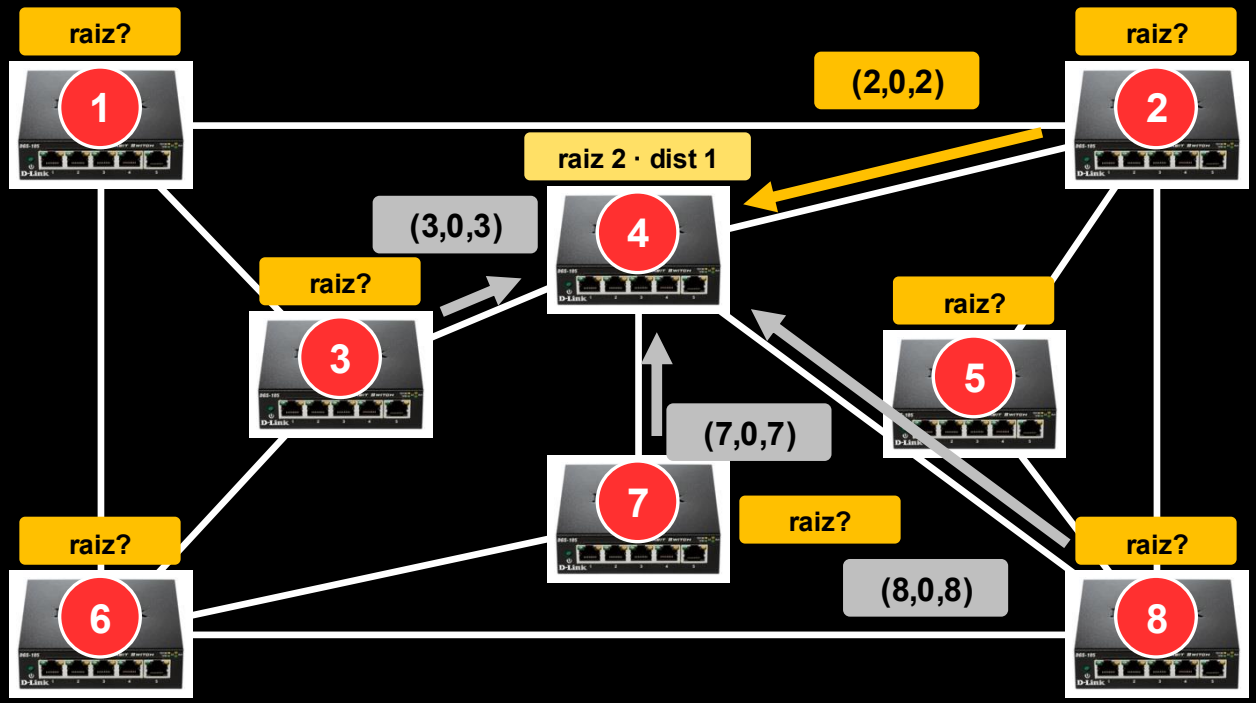


O switch 4 ouve:
(2,0,2) do 2
(3,0,3) do 3
(7,0,7) do 7
(8,0,8) do 8
a sua: (4,0,4)

Uma mensagem é melhor se:
1. indica um ID de raiz menor;
2. mesma raiz: distância mais curta;
3. mesma raiz e distância: ID do emissor menor.

Cada switch envia a sua mensagem por todas as portas. Vejamos de perto o switch 4: chega uma mensagem de cada um dos seus quatro vizinhos.

Regra 1: Ganha a Raiz Menor

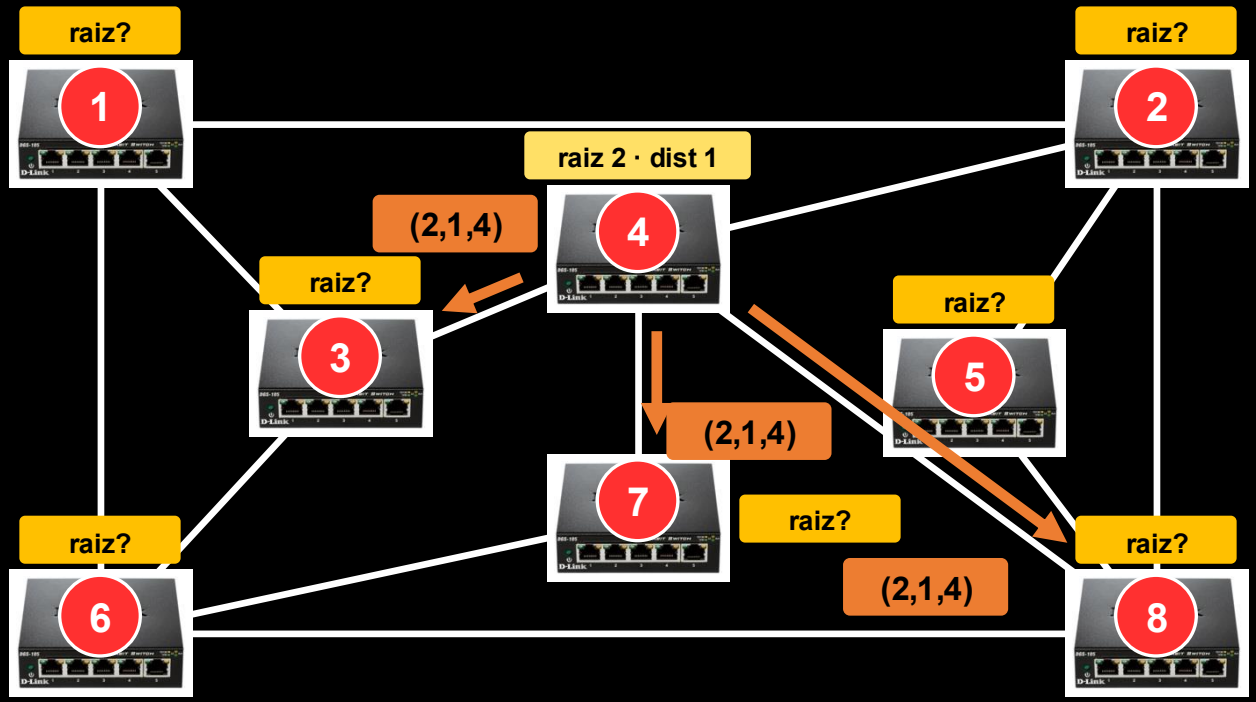


Comparar IDs de raiz:
2 < 3 < 4 < 7 < 8

O switch 4 acredita agora em:
raiz 2, distância 0 + 1 = 1, alcançada através do switch 2.

IDs de raiz em oferta: 2, 3, 7, 8 — e o do próprio 4. O menor é o 2, por isso o 4 deixa de afirmar que é a raiz e adota a oferta do 2, um salto mais longe.

O Switch 4 Passa a Mensagem

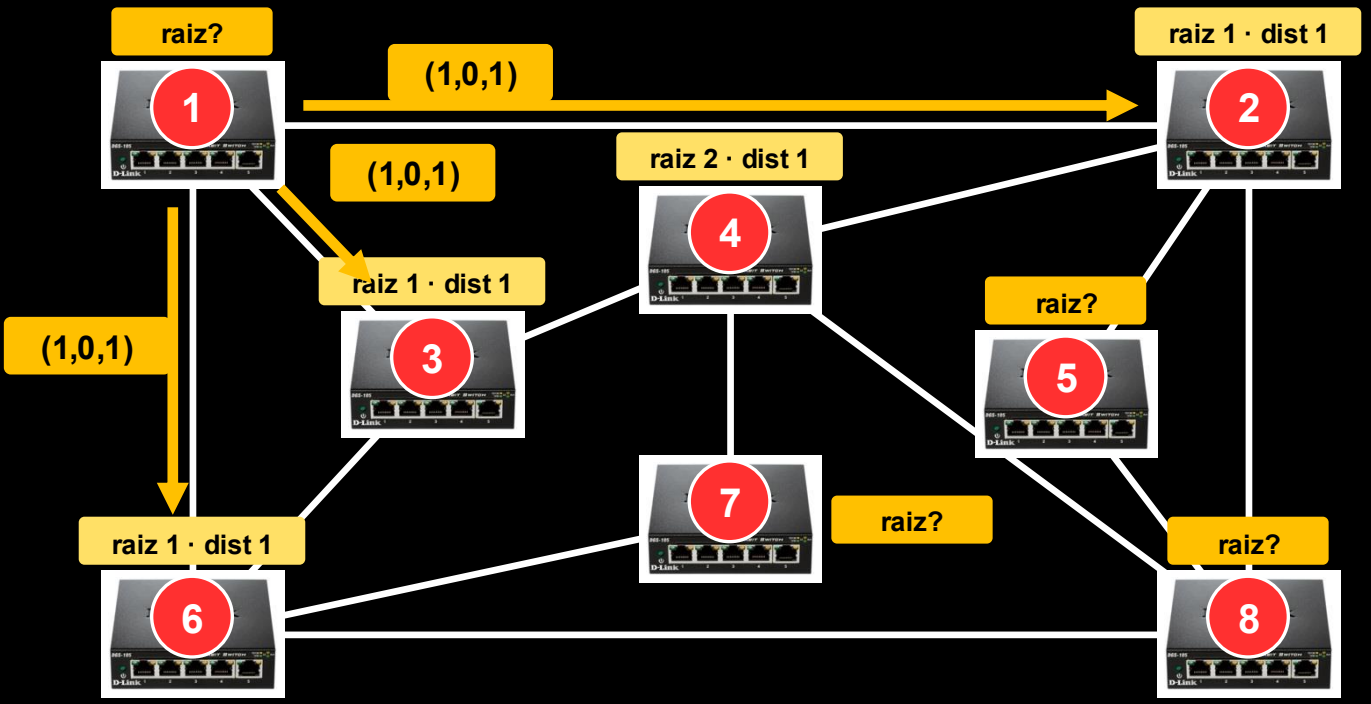


Um switch que ouve falar de uma raiz melhor deixa de originar mensagens.

Limita-se a retransmitir a melhor que conhece, com a sua própria distância e o seu próprio ID.

A partir de agora, o 4 anuncia a raiz em que acredita — (2, 1, 4) — nas suas outras portas, e nunca mais (4, 0, 4).

Entretanto, Junto ao Switch 1

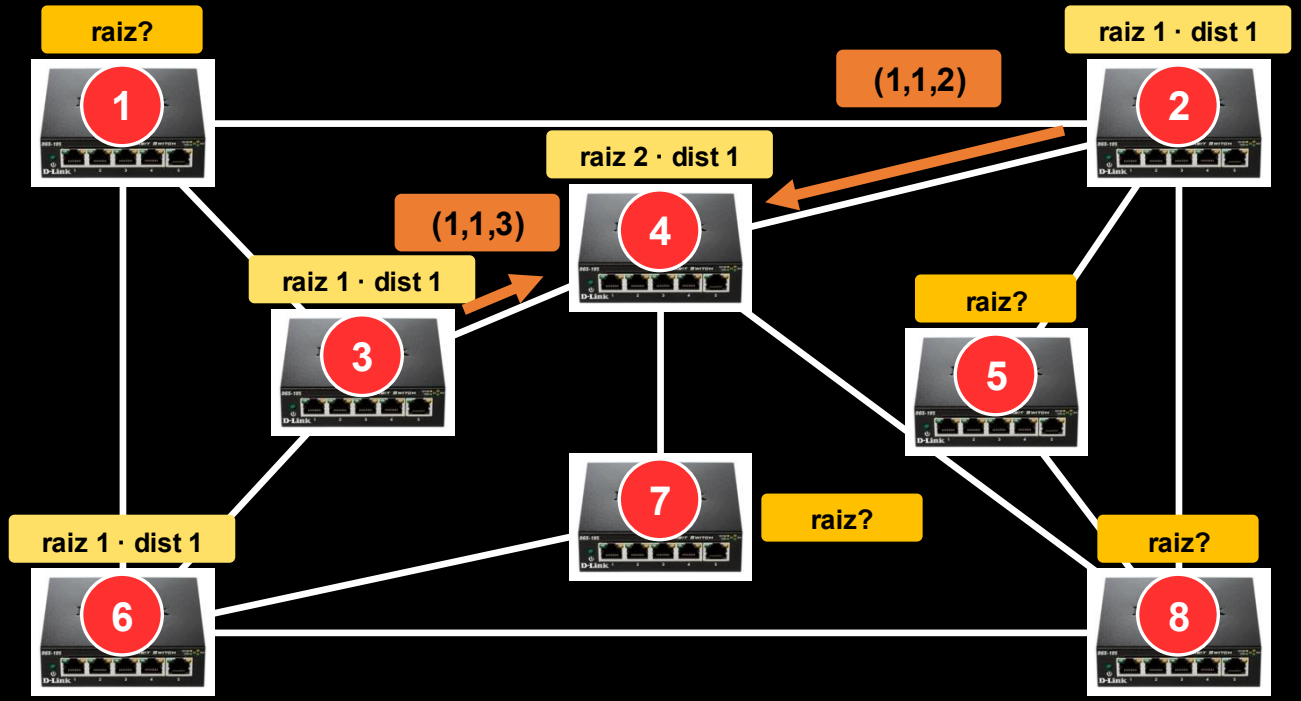


O switch 1 nunca ouve um ID menor — não existe nenhum menor — por isso continua a acreditar que é a raiz.

O 2, o 3 e o 6 adotam a raiz 1, à distância 1.

Ao mesmo tempo, o 2, o 3 e o 6 ouvem (1, 0, 1) do switch 1. Outra vez a regra 1: a raiz 1 bate tudo o que conheciam.

Melhores Ofertas Chegam ao 4

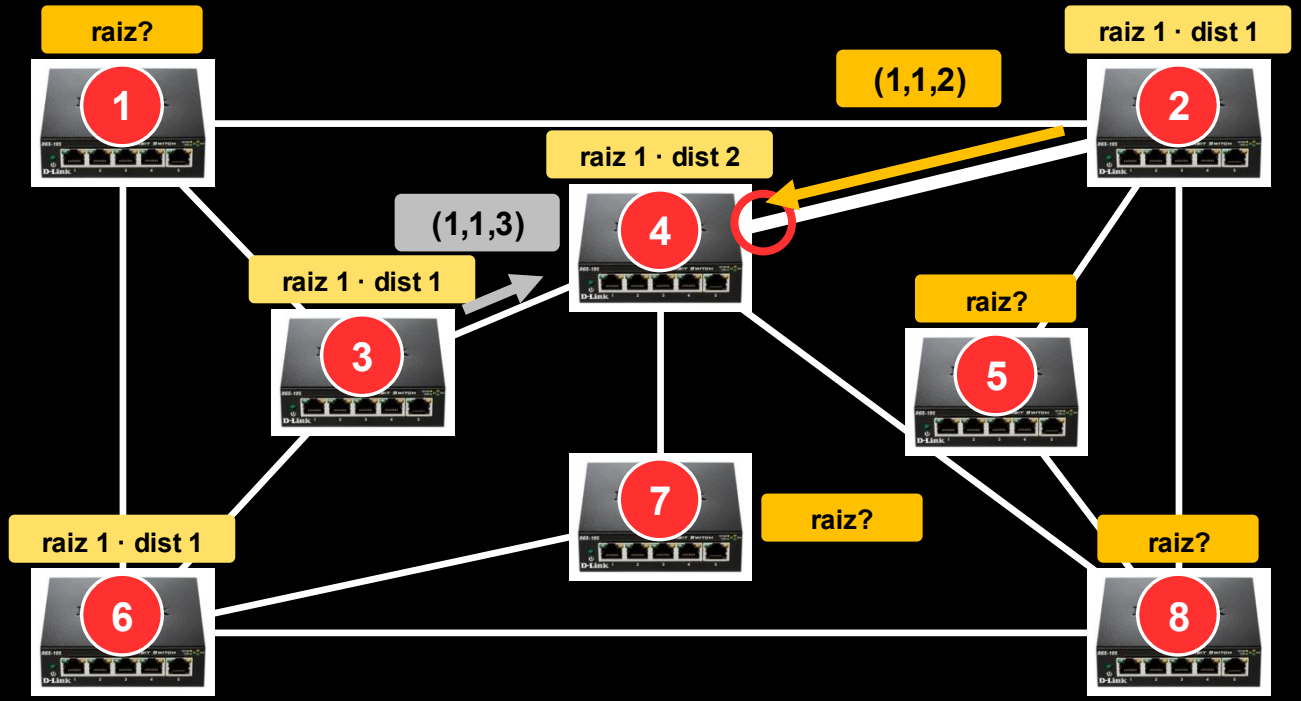


O switch 4 compara:
a sua crença atual: raiz 2
(1,1,2) do 2
(1,1,3) do 3

Regra 1: a raiz 1 bate a raiz 2.
Mas as duas novas ofertas empatam na regra 1…

O 2 e o 3 retransmitem agora a raiz 1 com a sua própria distância e ID: (1, 1, 2) e (1, 1, 3). Ambas indicam a raiz 1 — melhor do que a raiz 2 em que o 4 acreditava.

Um Empate: Decide a Regra 3

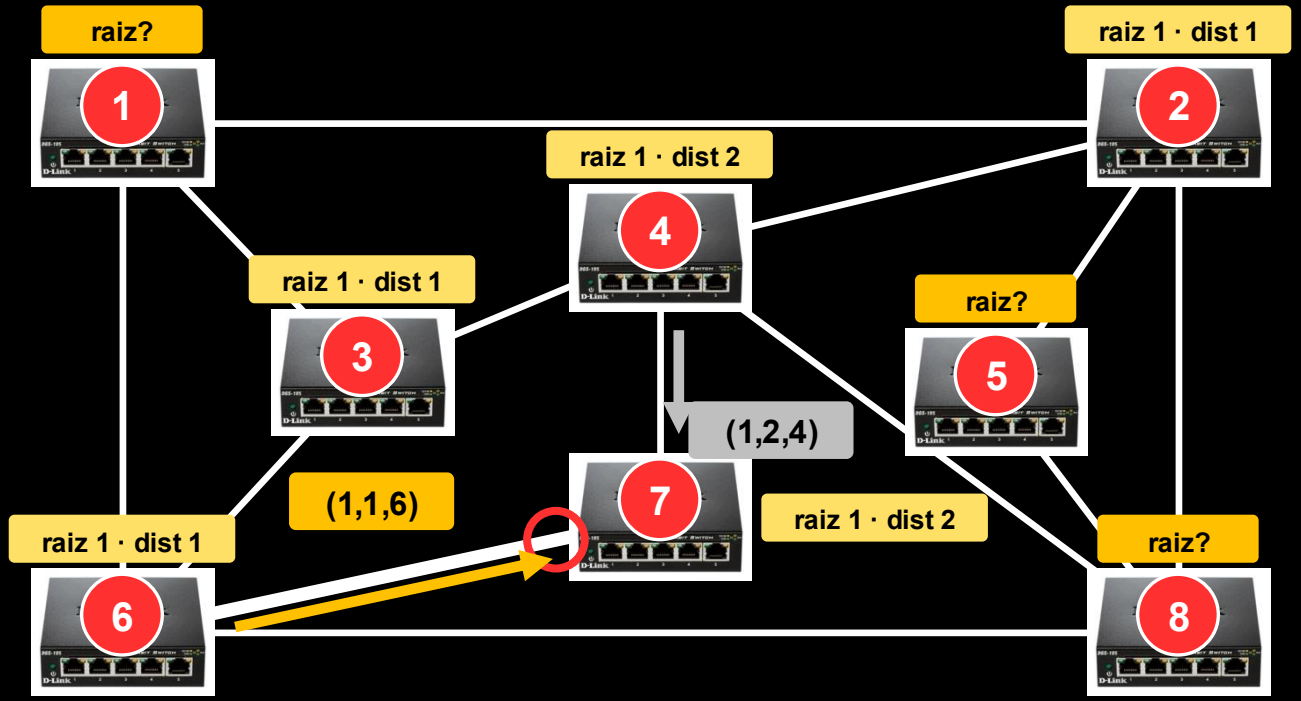


(1,1,2) vs (1,1,3)
regra 1: raiz 1 = raiz 1
regra 2: distância 1 = 1
regra 3: emissor 2 < 3

(1,1,2) ganha.
Círculo vermelho: porta raiz do 4.

Mesma raiz, mesma distância — só o emissor é diferente. Ganha o ID de emissor menor: o 2. A porta raiz do switch 4 é a que está virada para o 2, e a sua distância é 1 + 1 = 2.

A Regra 2 em Ação: o Switch 7

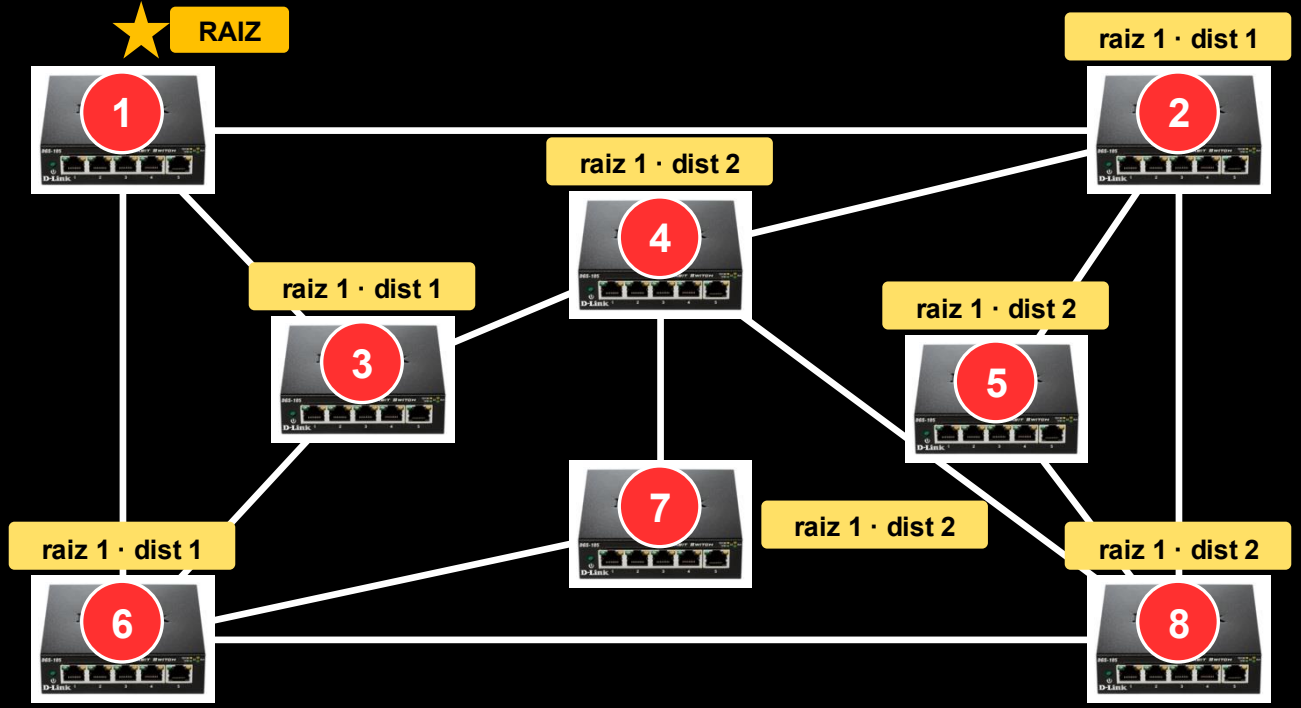


(1,1,6) vs (1,2,4)
regra 1: raiz 1 = raiz 1
regra 2: distância 1 < 2

(1,1,6) ganha.
A porta raiz do 7 está virada para o 6; a sua distância é 2.

O switch 7 ouve a raiz 1 vinda de ambos os vizinhos, mas a distâncias diferentes. Ganha a distância mais curta — não é preciso desempate — por isso o 7 chega à raiz através do 6.

Todos Convergem

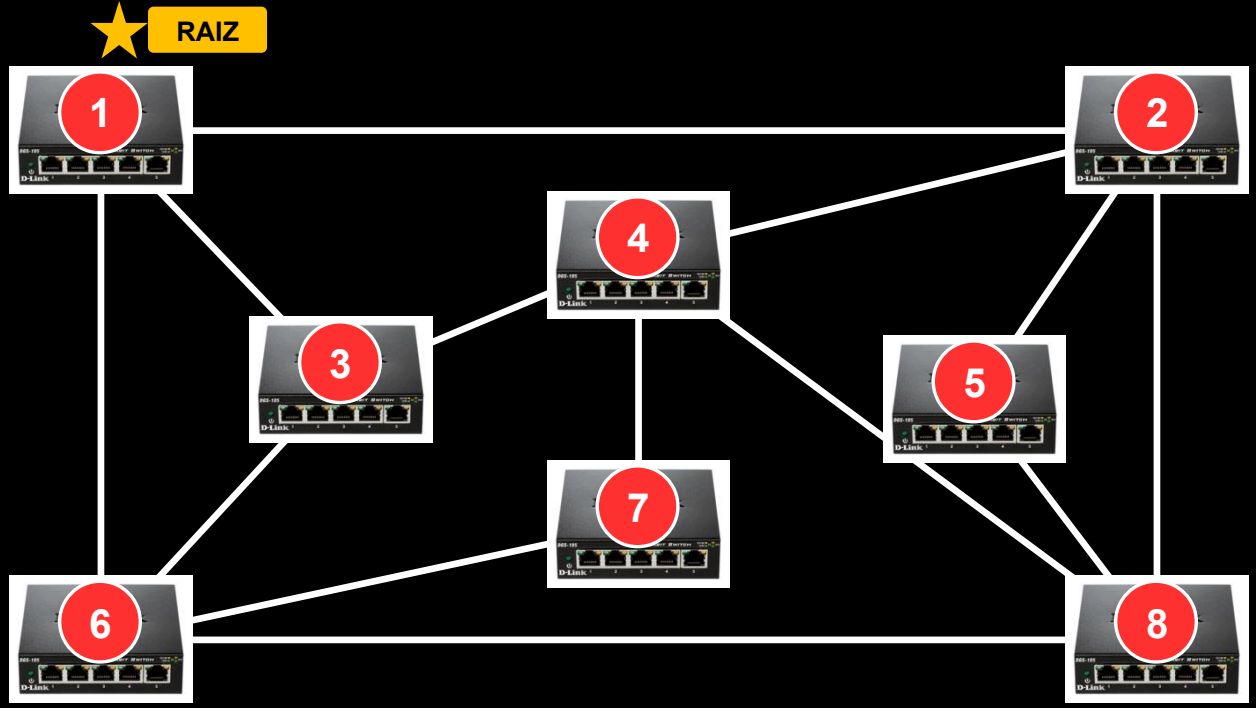


Só a raiz continua a originar mensagens; todos os outros switches retransmitem-nas, somando um à distância.

A seguir: a árvore que isto produz, passo a passo.

Algumas rondas depois, todos os switches acreditam na mesma raiz, conhecem a sua distância, e escolheram a sua porta raiz — usando apenas mensagens dos vizinhos diretos.

Ganha o ID Mais Baixo: o Switch 1

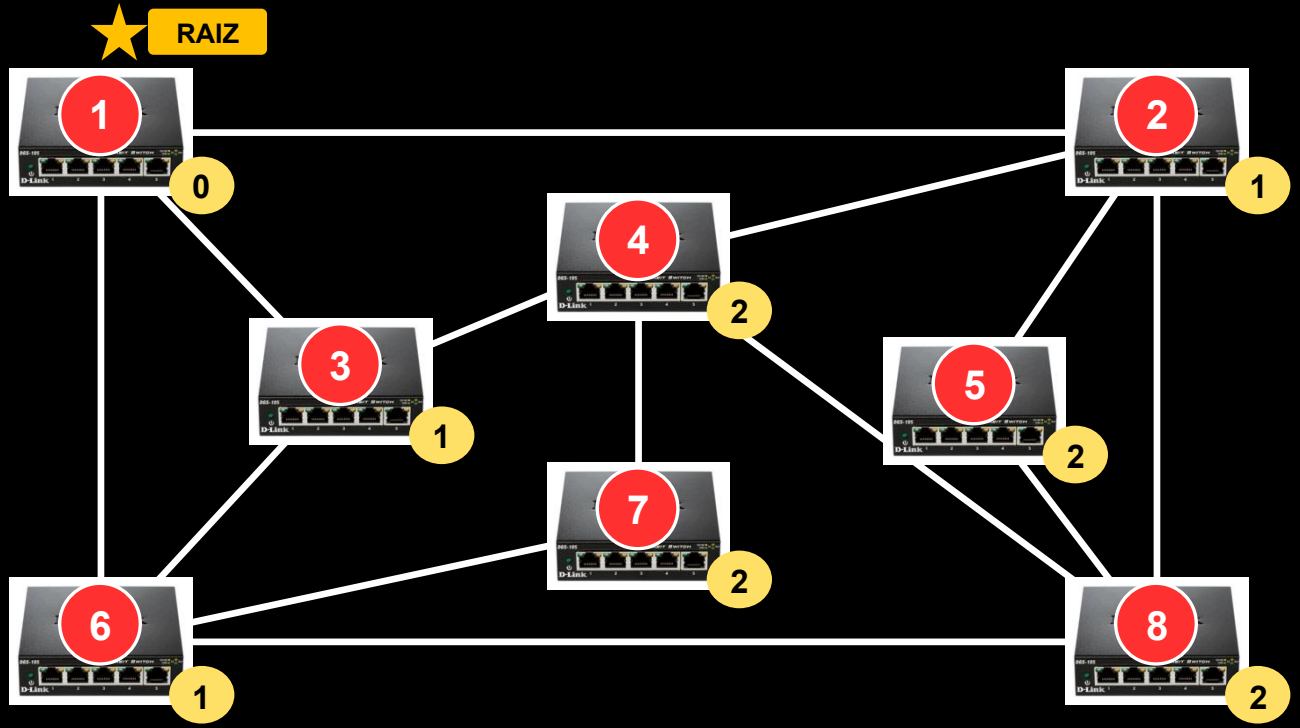


Regra 1:
uma mensagem ganha se indicar um ID de raiz menor.

Todos os switches acabam por ouvir falar do 1 — e deixam de acreditar em si próprios.

As mensagens que indicam uma raiz menor espalham-se salto a salto. Em poucas trocas, todos os switches concordam: o switch 1 tem o ID mais pequeno, por isso é a raiz.

A Que Distância da Raiz?

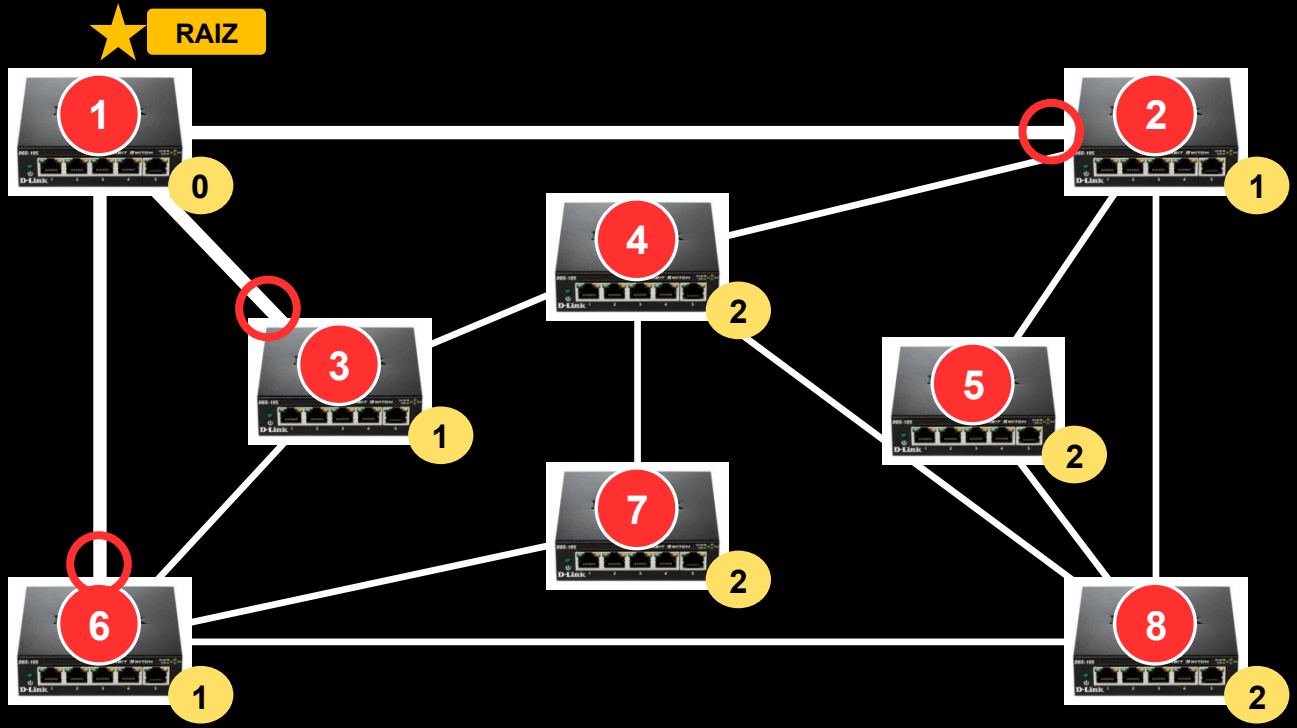


Números amarelos: distância em saltos até à raiz.

Switches 2, 3, 6: distância 1.
Switches 4, 5, 7, 8: distância 2.

Cada switch soma 1 à melhor distância que ouve. Os vizinhos da raiz estão a 1 salto; todos os outros, aqui, estão a 2 saltos.

Portas Raiz: Vizinhos do 1

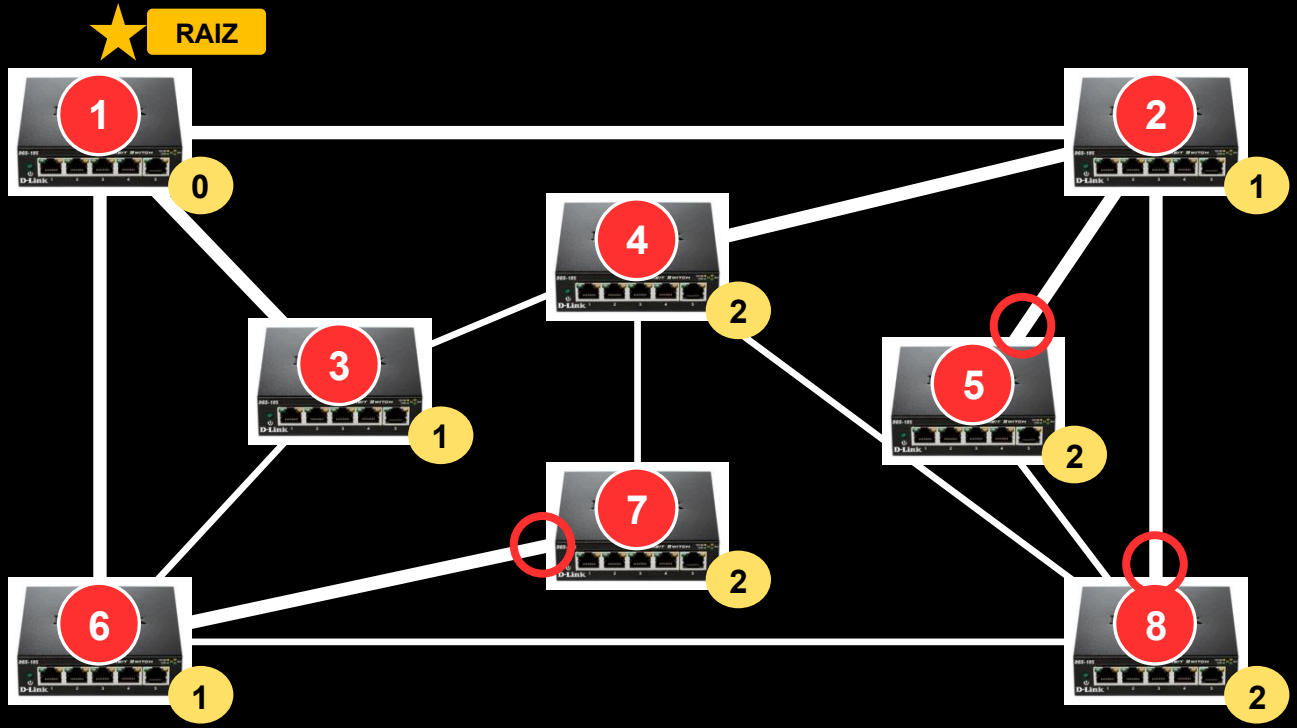


Círculo vermelho: uma porta raiz.

Linha grossa: uma ligação que a árvore mantém.

Todos os switches exceto a raiz escolhem uma porta raiz: a porta no seu melhor caminho para a raiz. Para o 2, o 3 e o 6 é simplesmente a ligação ao switch 1.

Portas Raiz do 5 e do 8

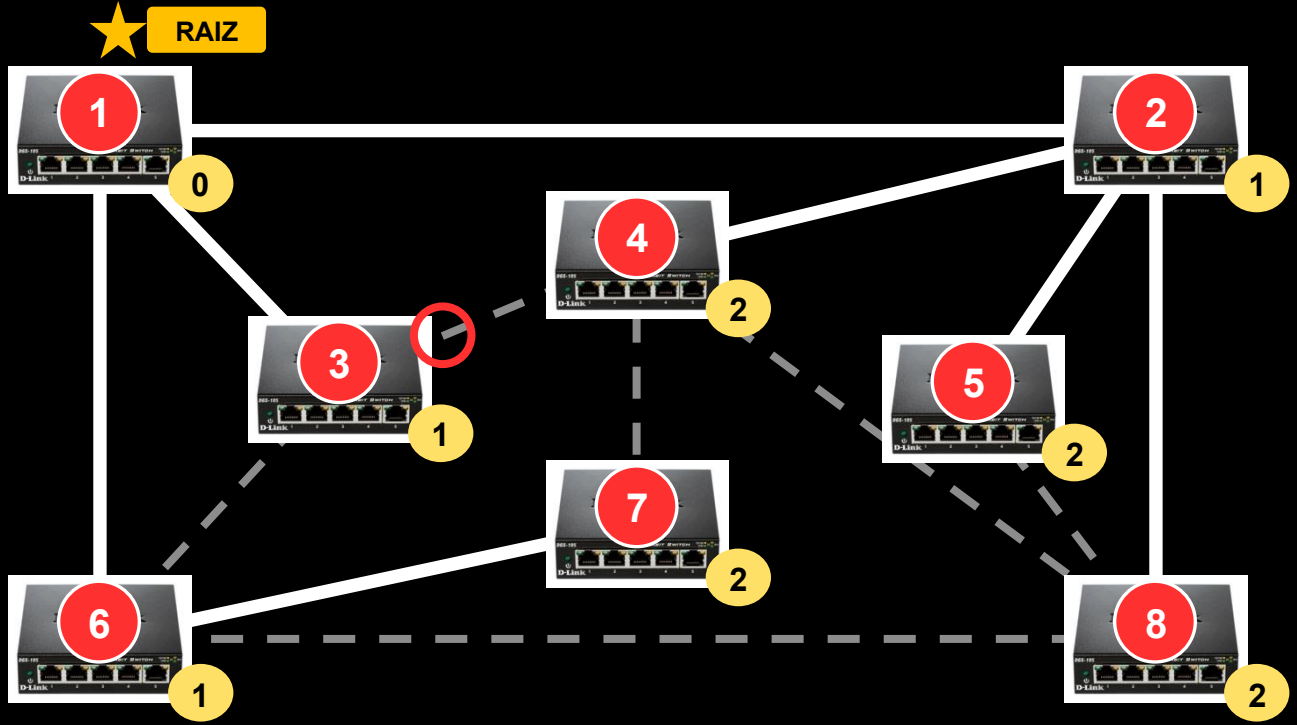


Switch 8:
ofertas do 2 (dist 1), do 6 (dist 1), do 4 e do 5 (dist 2).

Melhor: 2.

O switch 5 só tem um vizinho à distância 1: o 2. O switch 8 tem dois: o 2 e o 6 (o 4 e o 5 estão à distância 2). Mais uma vez ganha o ID mais baixo: o 2. Com o 4 (via 2) e o 7 (via 6) já resolvidos, todos os switches têm agora a sua porta raiz.

Um Encaminhador por Ligação

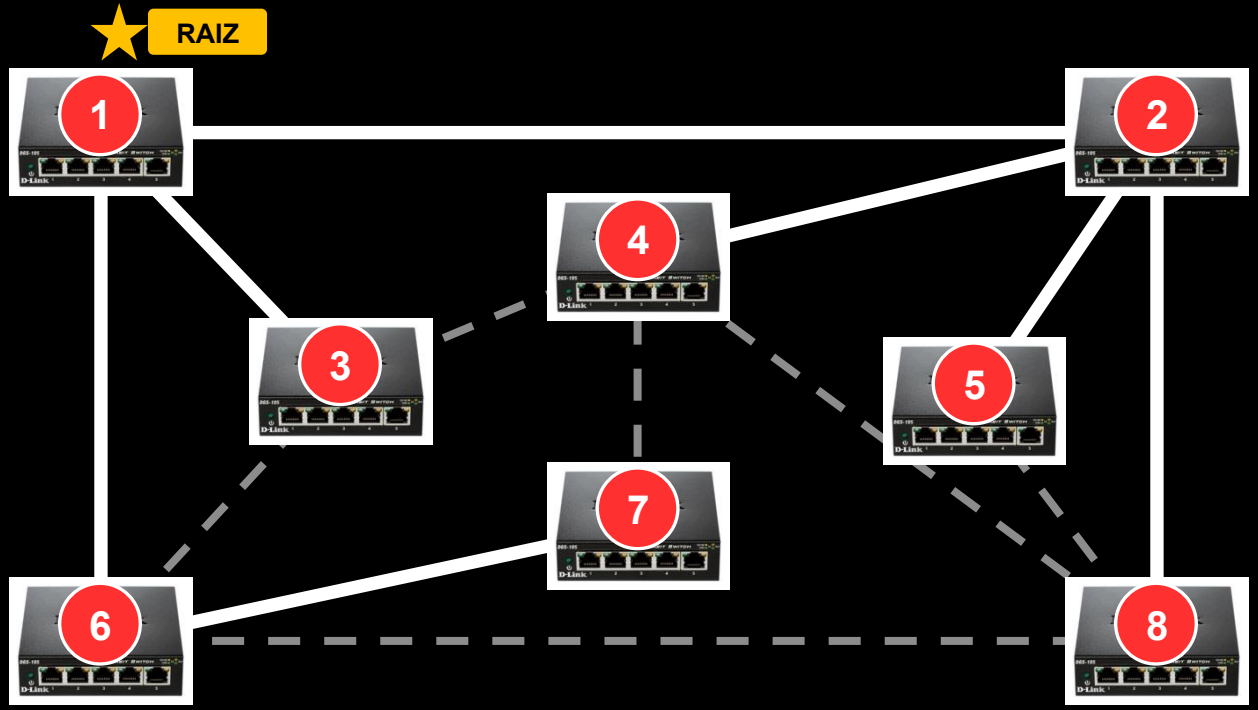


Círculo vermelho: porta designada do switch 3 na ligação 3–4: ativa, usada para inundar tramas.

a ponta do 4 nessa ligação: bloqueada.

Em cada ligação restante, a ponta mais próxima da raiz continua a encaminhar (a sua porta designada); a outra ponta bloqueia. Na ligação 3–4, o switch 3 (distância 1) bate o 4 (distância 2). Mesma distância? Ganha o ID mais baixo (e entre duas ligações paralelas para o mesmo switch, o número de porta mais baixo).

Resultado: Árvore sem Ciclos

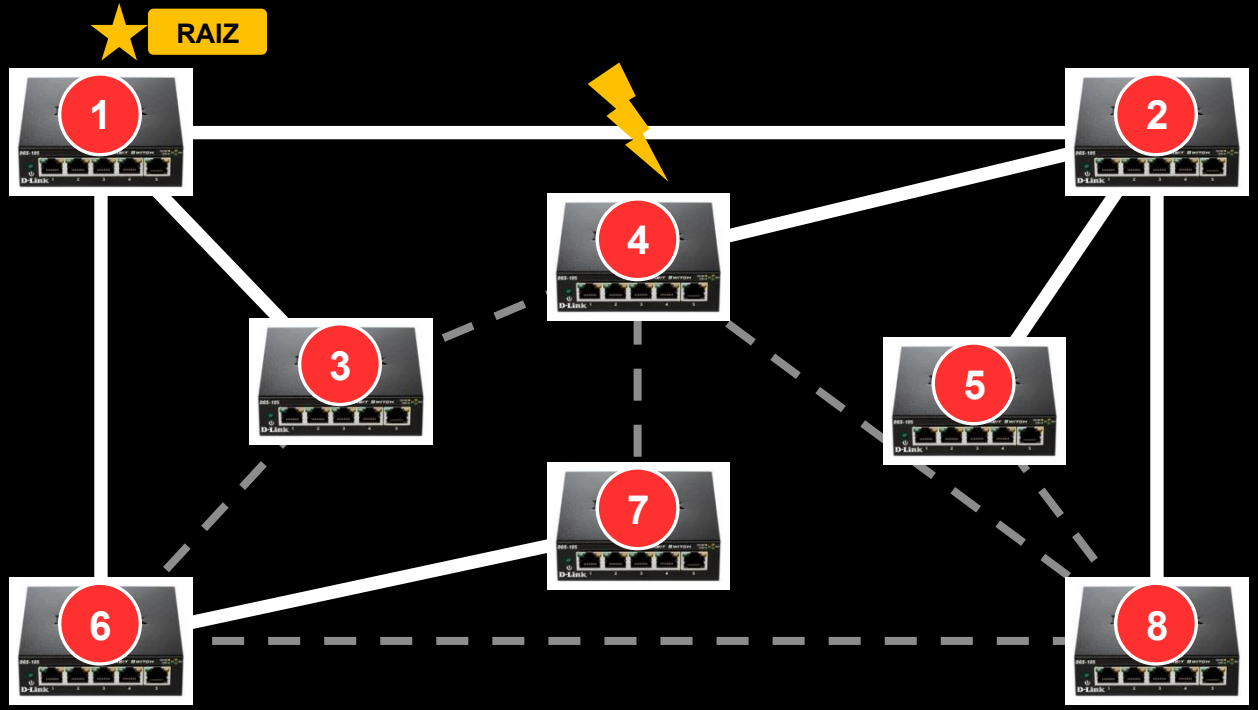


Ativas:
1–2, 1–3, 1–6, 2–4, 2–5, 2–8, 6–7

Bloqueadas (tracejado):
3–4, 3–6, 4–7, 4–8, 5–8, 6–8

Sete ligações ativas ligam os oito switches com exatamente um caminho entre quaisquer dois. As outras seis continuam ligadas, apenas não encaminham.

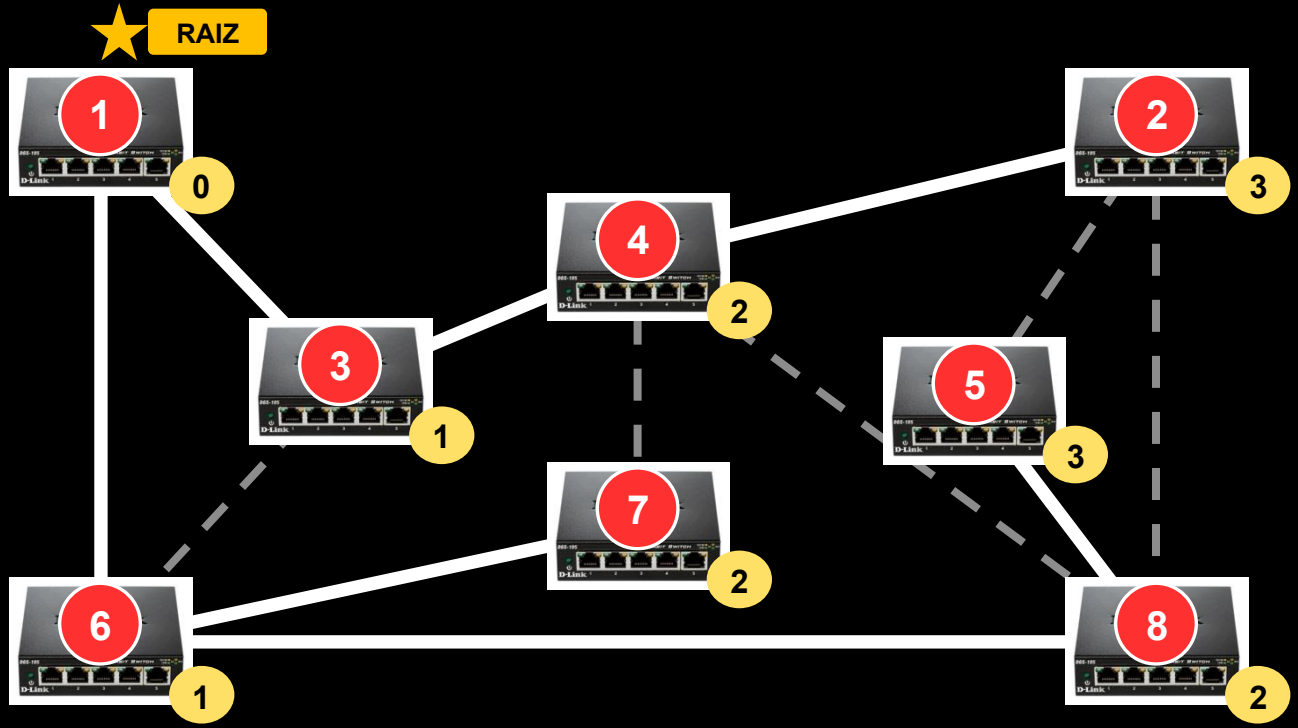
Uma Ligação Falha



A raiz continua a enviar as suas mensagens periódicas — mas elas já não chegam ao 2, 4, 5 ou 8 através de 1–2.

A ligação entre os switches 1 e 2 parte-se. Era a porta raiz do switch 2 — e os switches 4, 5 e 8 chegavam à raiz através do 2.

A Árvore Volta a Formar-se



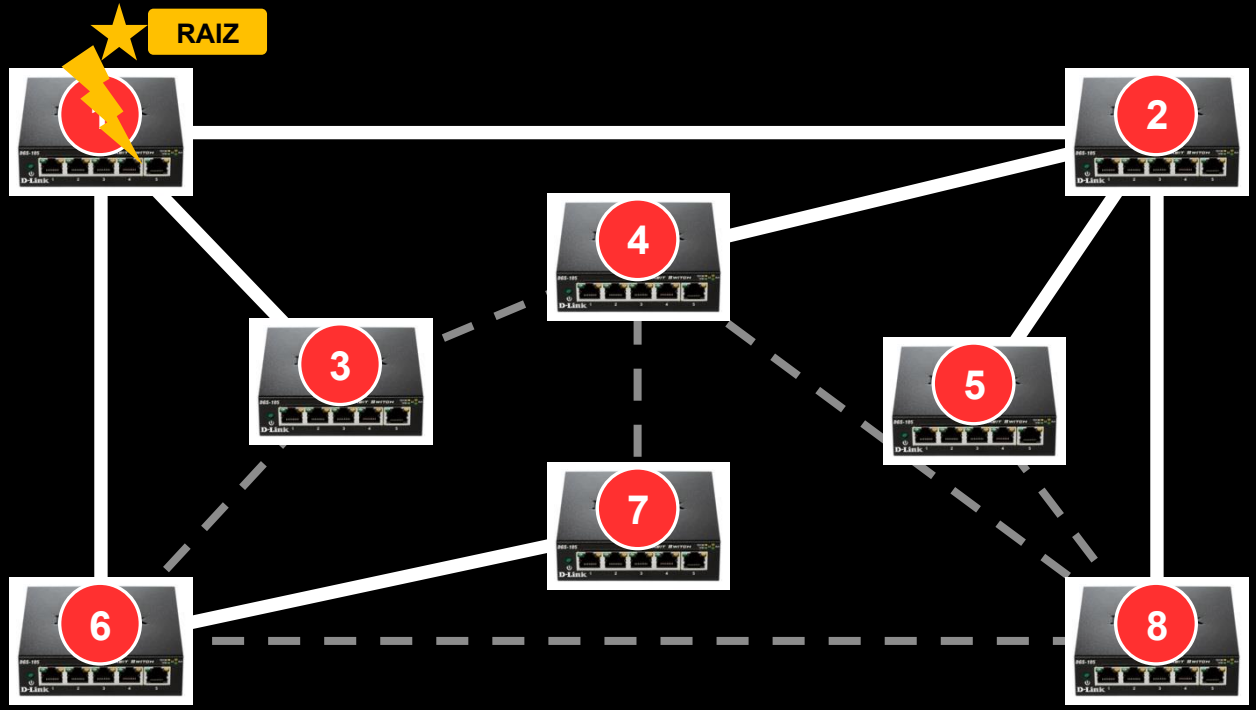
Reativadas:
3–4, 5–8, 6–8

Agora bloqueadas:
2–5, 2–8

O switch 2 está agora a 3 saltos da raiz.

As mesmas regras correm de novo: o 4 chega agora à raiz através do 3, o 8 através do 6, o 2 através do 4 e o 5 através do 8. Três ligações bloqueadas acordam; 2–5 e 2–8 ficam de reserva.

E Se a Própria Raiz Falhar?

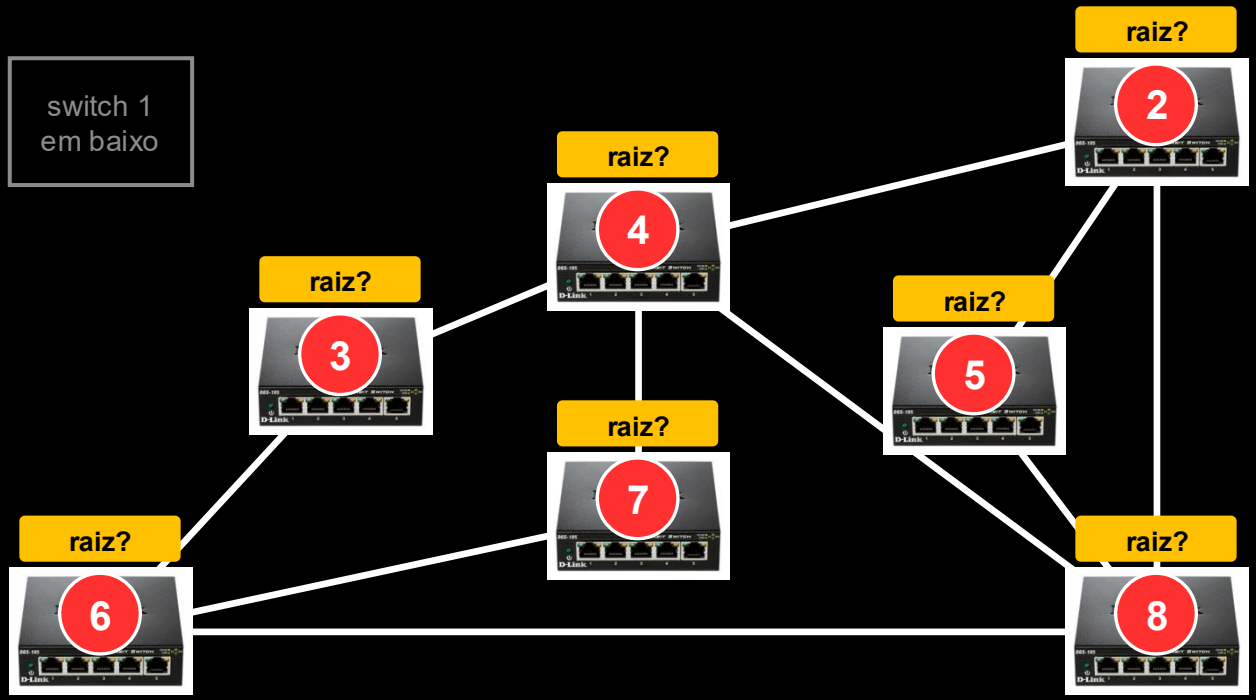


A raiz era o único switch a gerar mensagens de configuração.

Quem dá por isso — e como?

De volta à árvore original. Desta vez é o switch 1 que morre — a raiz, e com ela todas as ligações que tinha.

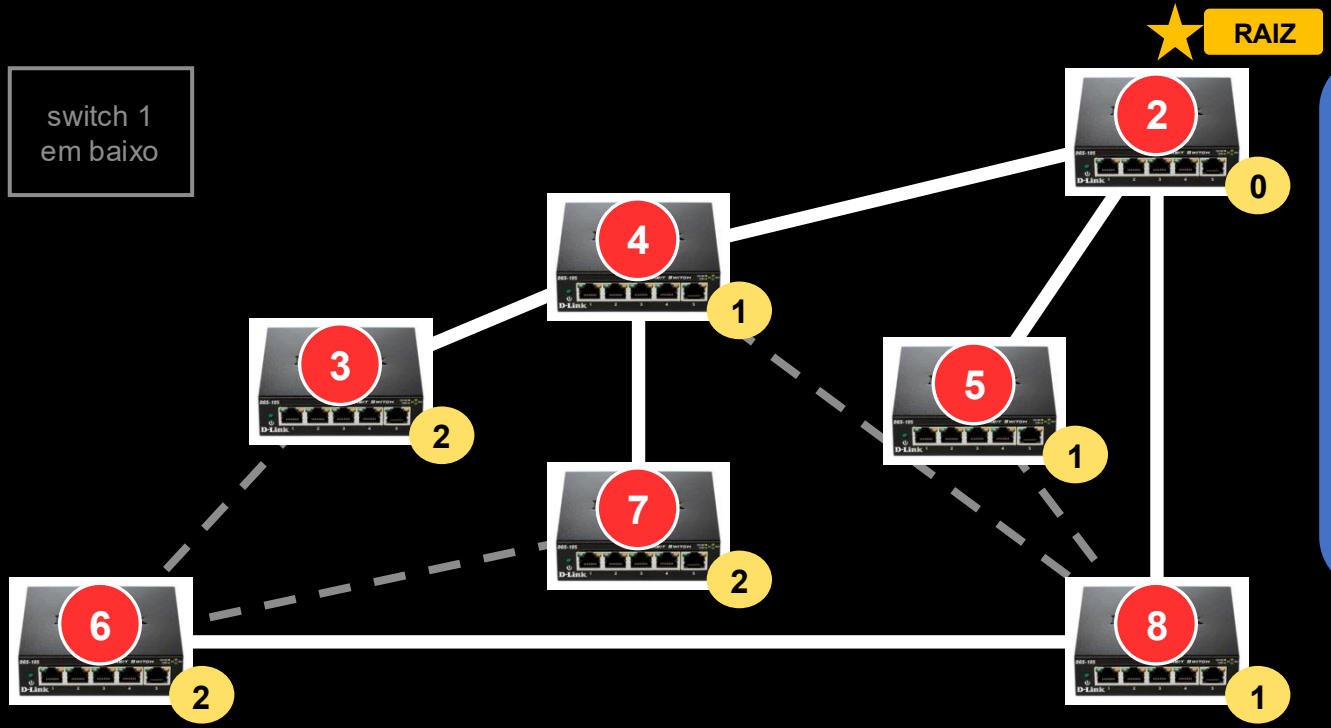
Silêncio, Depois uma Nova Eleição



Os vizinhos da falha veem a ligação cair; para os outros, só um timeout revela que a raiz desapareceu.

Sem alarme, sem notificação: depois de vários períodos sem mensagens da raiz, cada switch esgota o seu temporizador e volta a afirmar que é a raiz — exatamente como ao ligar.

Uma Nova Raiz, uma Nova Árvore



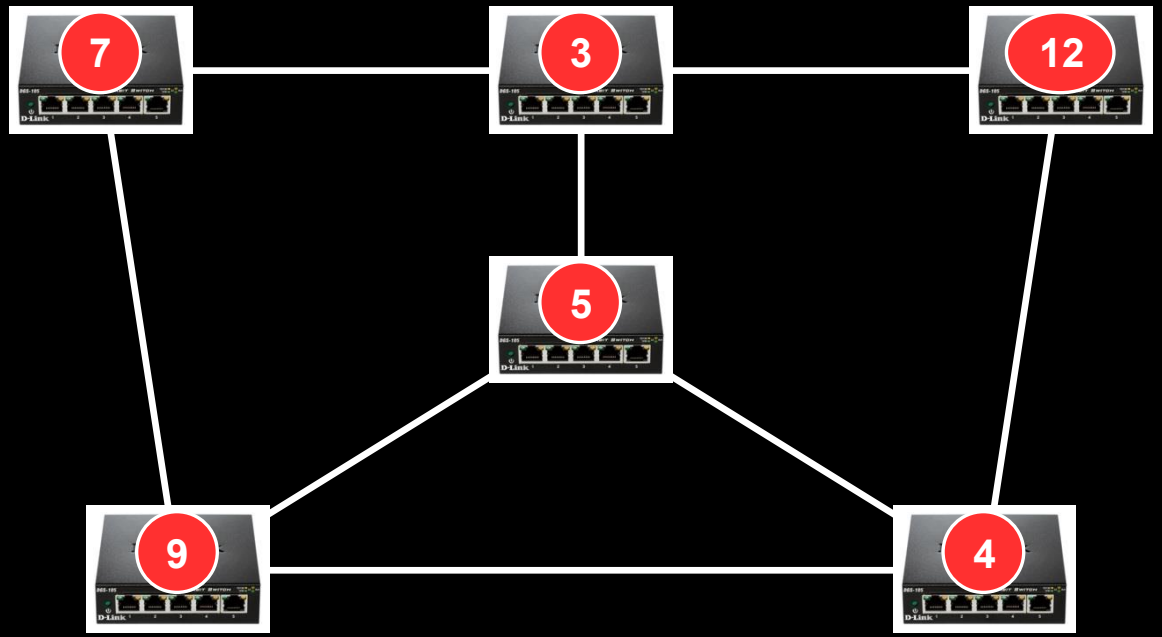
Ativas:
2–4, 2–5, 2–8, 3–4, 4–7, 6–8

Bloqueadas:
3–6, 4–8, 5–8, 6–7

Ninguém configurou nada.

A mesma eleição corre entre os sobreviventes: o switch 2 tem agora o ID mais pequeno, por isso torna-se a raiz, e todas as portas raiz e ligações bloqueadas são recalculadas.

Exercício: Construam a Árvore

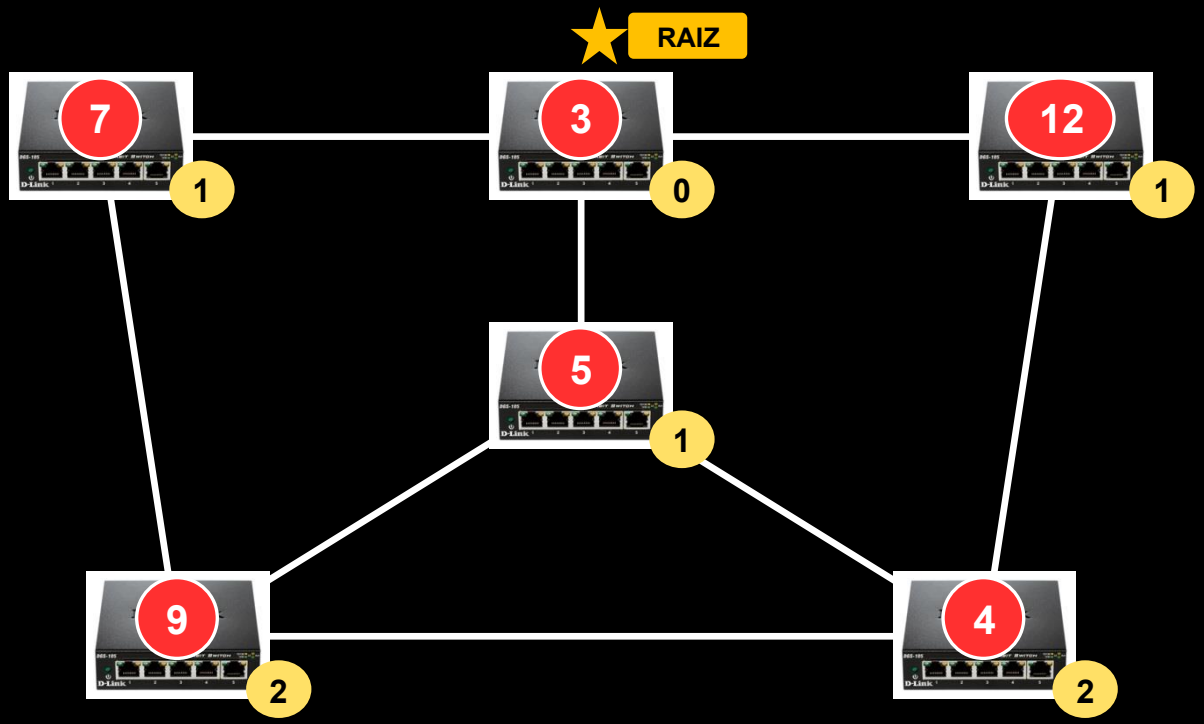


A vossa vez:
Seis switches; os números são os seus IDs.

1. Que switch se torna a raiz?
2. Qual é a porta raiz de cada switch?
3. Que ligações acabam bloqueadas?

Cuidado: dois switches enfrentam um empate.

Resposta: Raiz e Distâncias

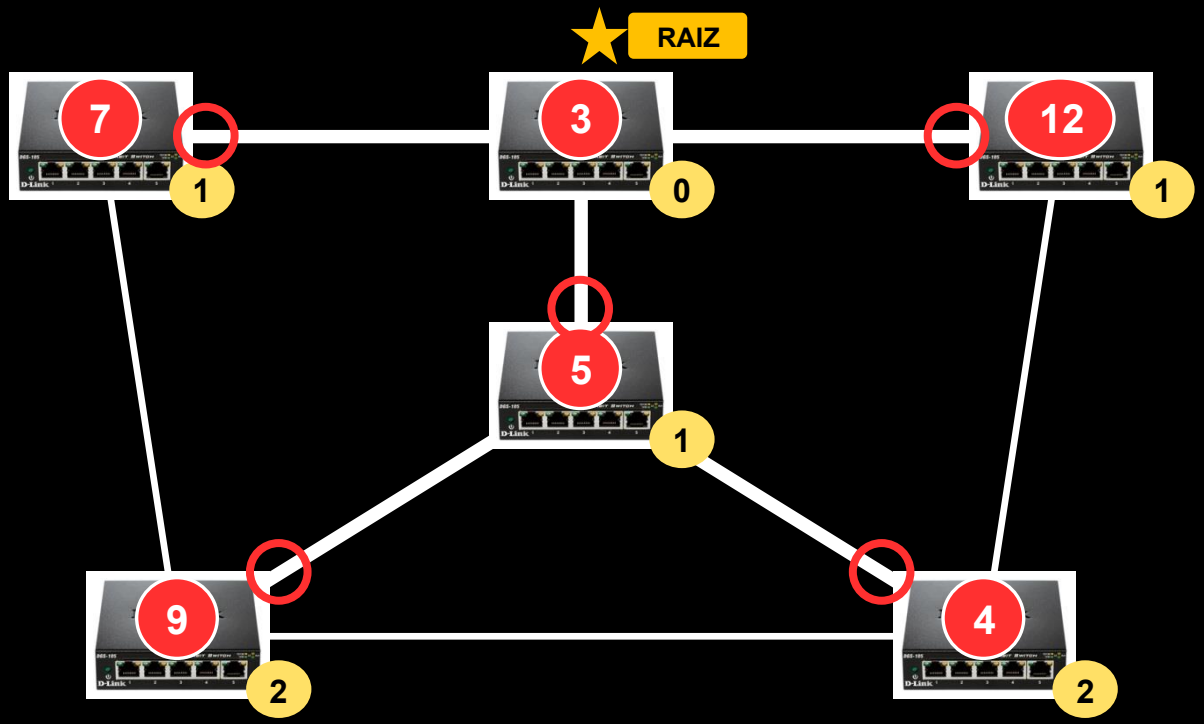


Raiz: switch 3.

Não é o switch desenhado ao centro, nem o que tem mais ligações — é o de ID mais pequeno.

O switch 3 tem o ID mais pequeno. Os seus vizinhos 7, 5 e 12 estão a 1 salto; o 9 e o 4 estão a 2 saltos.

Resposta: as Portas Raiz

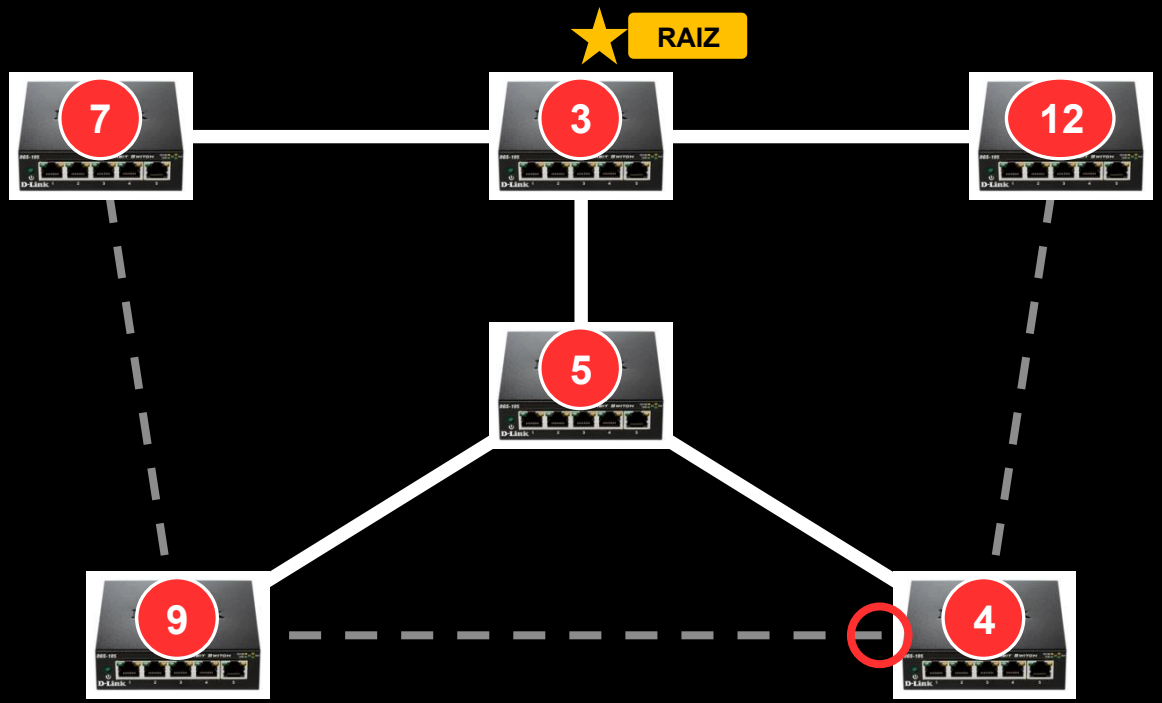


Os dois empates: mesma raiz, mesma distância → ID do emissor menor.

9 → via 5 (não 7)
4 → via 5 (não 12)

O 7, o 5 e o 12 apontam diretamente para o 3. O switch 9 empata entre o 7 e o 5 (ambos à distância 1): ganha o 5. O switch 4 empata entre o 5 e o 12: ganha o 5.

Resposta: as Ligações Bloqueadas



Bloqueadas:
7–9, 12–4, 9–4

Círculo vermelho: o 4 é a ponta designada da ligação 9–4 — distância igual, ID mais baixo.

Ligação 7–9: o 7 está mais perto, por isso o 9 bloqueia. Ligação 12–4: o 12 está mais perto, por isso o 4 bloqueia. Ligação 9–4: distância igual — o ID mais baixo, o 4, é o designado e o 9 bloqueia.

Spanning Tree Protocol

Conclusão:
o STP permite aos switches acordarem, sem coordenador central, numa única árvore sem ciclos que cobre toda a rede — eleger uma raiz, calcular distâncias, manter um caminho, bloquear o resto (os switches reais usam custos por ligação, e IDs compostos por uma prioridade mais um endereço MAC). Nada é alguma vez desligado fisicamente, por isso a fiabilidade nunca se perde realmente, fica apenas de reserva — e se a própria raiz for abaixo, a mesma eleição corre de novo e a rede recupera sozinha.

Um Pouco de História: Perlman

- **O algoritmo é dela:** Radia Perlman desenhou o algoritmo de spanning tree na Digital Equipment Corporation e publicou-o em 1985 — o IEEE 802.1D normalizou-o em 1990, e quase todos os switches vendidos hoje ainda correm um descendente dele.

- **Também o escreveu como poema:** depois de desenhar o algoritmo, resumiu-o em “Algorhyme”, publicado com o seu artigo de 1985.

- **Não foi a sua única invenção em redes:** mais tarde esteve na origem do TRILL (IETF, 2011), um protocolo pensado para contornar os limites de escala do próprio Spanning Tree em redes grandes.

Um Pouco de História: Perlman

- **O alg** erlman
  span ment (1985
  — o l -o em ches
  vendi m des(
- **Tamb poema** ritmo,
  resun ublicac
- **Não f o em r**
  origer), um p rnar
  os lim o Spar .



Perlman published a poem on STP, called "Algorhyme":[22]

I think that I shall never see
A graph more lovely than a tree.

A tree whose crucial property
Is loop-free connectivity.

A tree which must be sure to span
So packets can reach every LAN.

First the root must be selected.
By ID it is elected.

Least cost paths from root are traced.
In the tree these paths are placed.

A mesh is made by folks like me
Then bridges find a spanning tree.

— Radia Perlman, "Algorhyme"

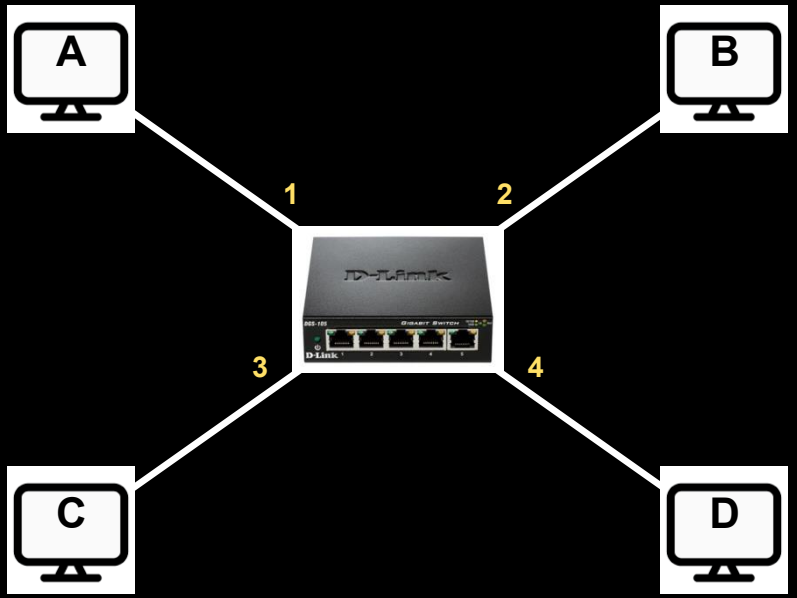
O Rapid Spanning Tree Hoje

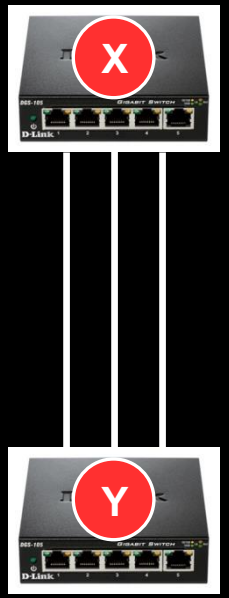
- **O STP clássico é lento:** uma mudança de topologia pode levar dezenas de segundos a reconvergir: os switches esperam por temporizadores conservadores antes de confiar na nova árvore.

- **O RSTP (IEEE 802.1w) resolve isto:** as mesmas regras de eleição, mas com handshakes explícitos em vez de temporizadores — cerca de um segundo em vez de dezenas.

- **É o que corre hoje:** o RSTP e as suas variantes estão muito difundidos e vêm muitas vezes ativos por omissão — o STP clássico é hoje sobretudo uma base de ensino.

Exercício: Inundar ou Encaminhar?

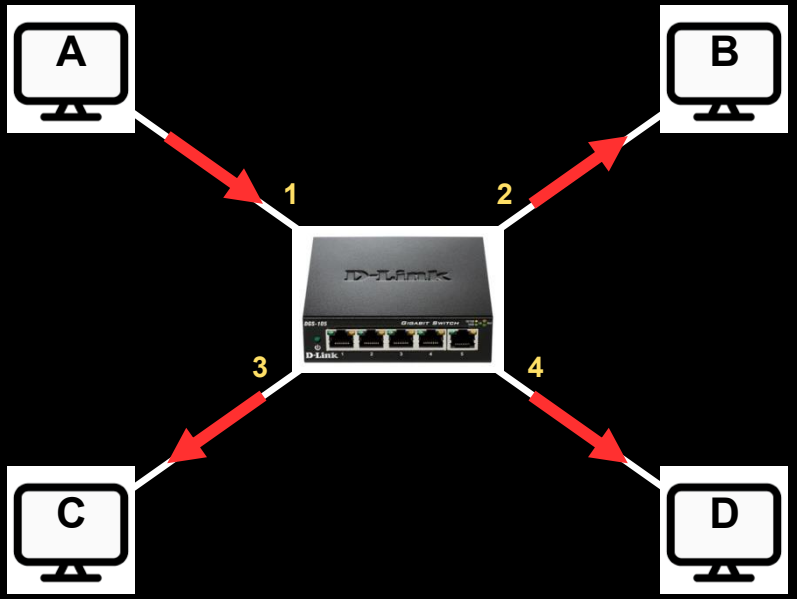




A vossa vez:
Esquerda: a tabela do switch começa vazia.
1) A envia para D.
2) D responde a A.
3) B envia para A.
A trama 3 é inundada, ou encaminhada por uma única porta?

Direita: três ligações paralelas unem X e Y. Quantas mantém ativas a spanning tree?

Trama 1: A para D



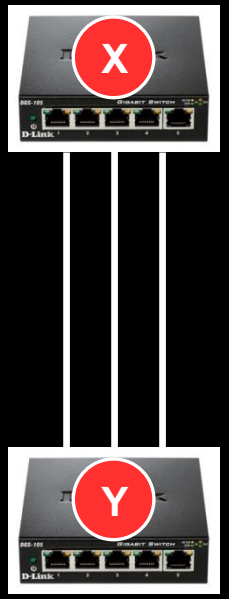
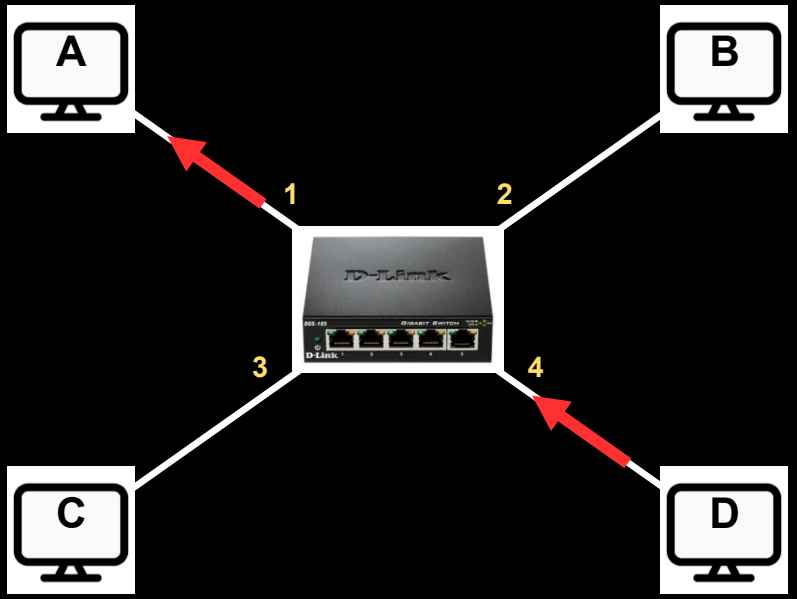


Tabela do switch

Porta	Endereço
1	A

O switch aprende A na porta 1. D é desconhecido, por isso a trama é inundada pelas portas 2, 3 e 4.

Trama 2: D Responde a A



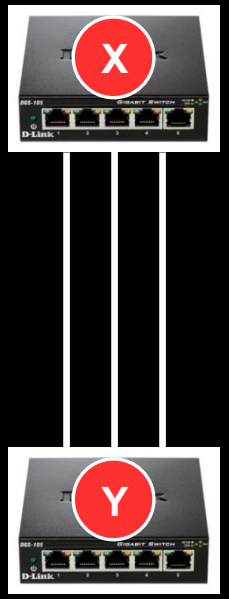
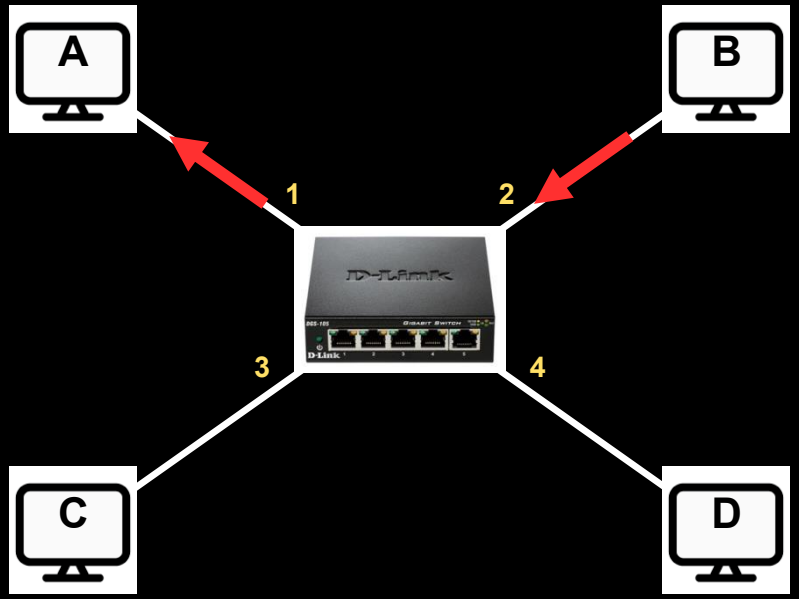


Tabela do switch

Porta	Endereço
1	A
4	**D**

O switch aprende D na porta 4. A já é conhecido, por isso a resposta sai apenas pela porta 1.

Trama 3: B para A — Encaminhada



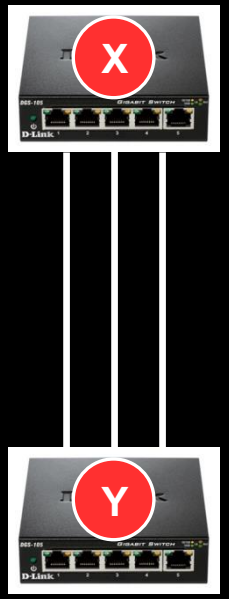
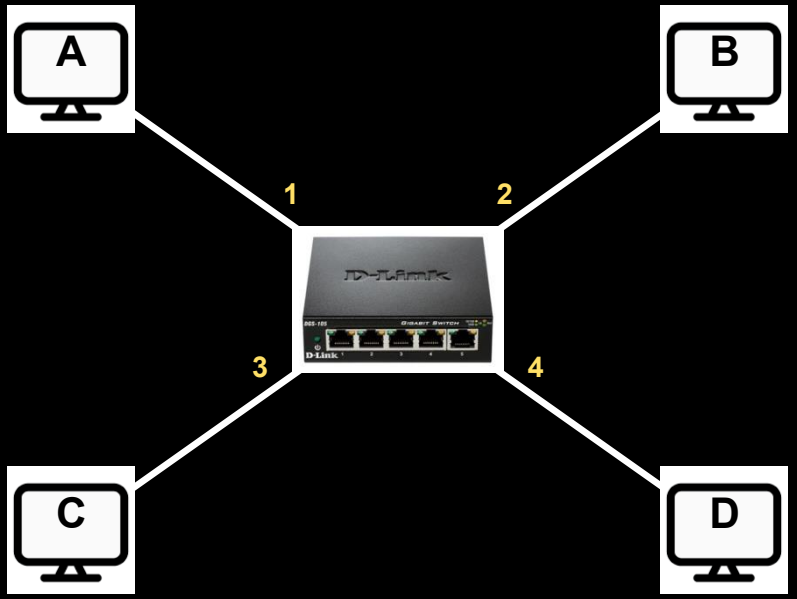


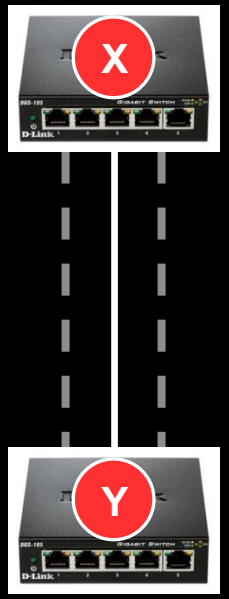
Tabela do switch

Porta	Endereço
1	A
4	D
2	**B**

O switch aprende B na porta 2 e encaminha a trama apenas pela porta 1: A foi aprendido no momento em que enviou a trama 1.

Três Ligações, Sobrevive Uma





Por muitas ligações diretas que acrescentem entre dois switches, a árvore mantém exatamente um caminho ativo entre eles.

Entre X e Y todas as ligações são igualmente boas. Exatamente uma fica ativa — os empates entre ligações para o mesmo switch são desfeitos pelo número de porta — as outras duas bloqueiam.

Um Endereço por Trama?

Pergunta:
até agora, todos os switches olharam para um endereço de destino completo em cada trama. Transportar o endereço inteiro, sempre, é a única forma de uma rede comutada funcionar?

Comutação de Pacotes

- **O que já conhecem:** aquilo a que uma aula anterior chamou comutação de pacotes: cada trama transporta um endereço de destino completo; cada switch decide, de forma independente, salto a salto, sem qualquer fase de coordenação ou estabelecimento de caminho.

- **A vantagem:** não há estado de ligação a manter — se um switch ou uma ligação morrer, as tramas seguem os novos caminhos assim que a rede os tiver reaprendido.

- **O custo:** esse endereço completo tem de ser transportado, e procurado, em cada trama —sempre, não apenas uma vez.

Comutação por Circuitos Virtuais

- **A ideia:** antes de quaisquer dados circularem, uma mensagem de estabelecimento percorre o caminho uma vez, e cada switch ao longo dele guarda uma pequena entrada numa tabela só para esta ligação (ou, de forma mais geral, para este fluxo de dados).

- **O que as tramas de dados transportam depois:** não um endereço completo — um pequeno Virtual Circuit Identifier (VCI), único apenas na ligação em que viaja.

- **O trabalho de cada switch:** procurar o VCI de entrada, trocá-lo pelo VCI de saída que essa ligação espera, encaminhar.

Circuito Virtual: Estabelecimento

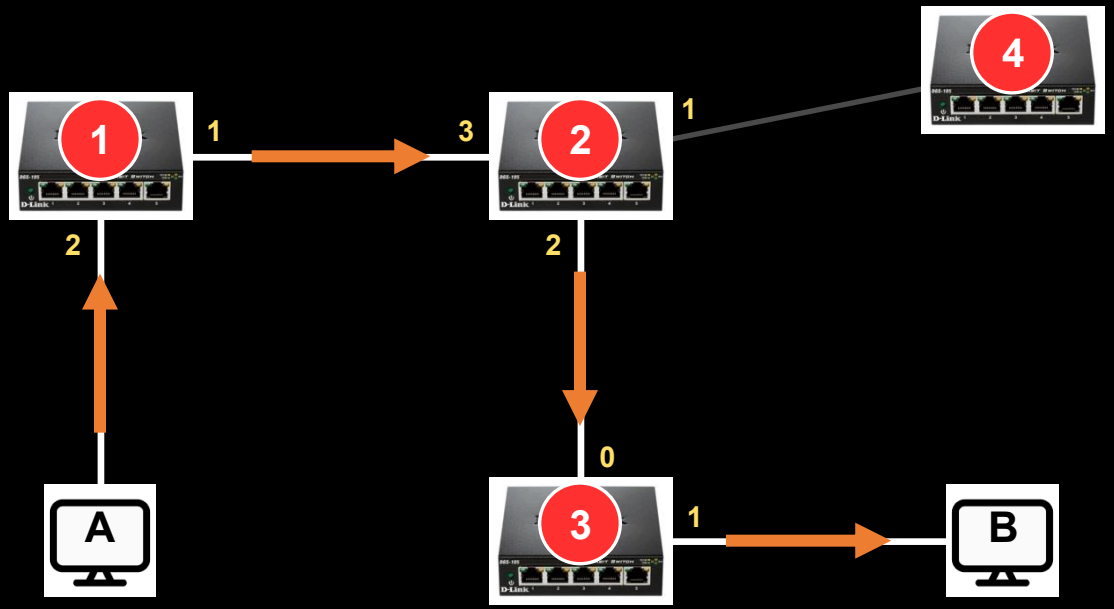


Tabela VC, switch 1

Porta ent.	VCI ent.	Porta saída	VCI saída

Tabela VC, switch 2

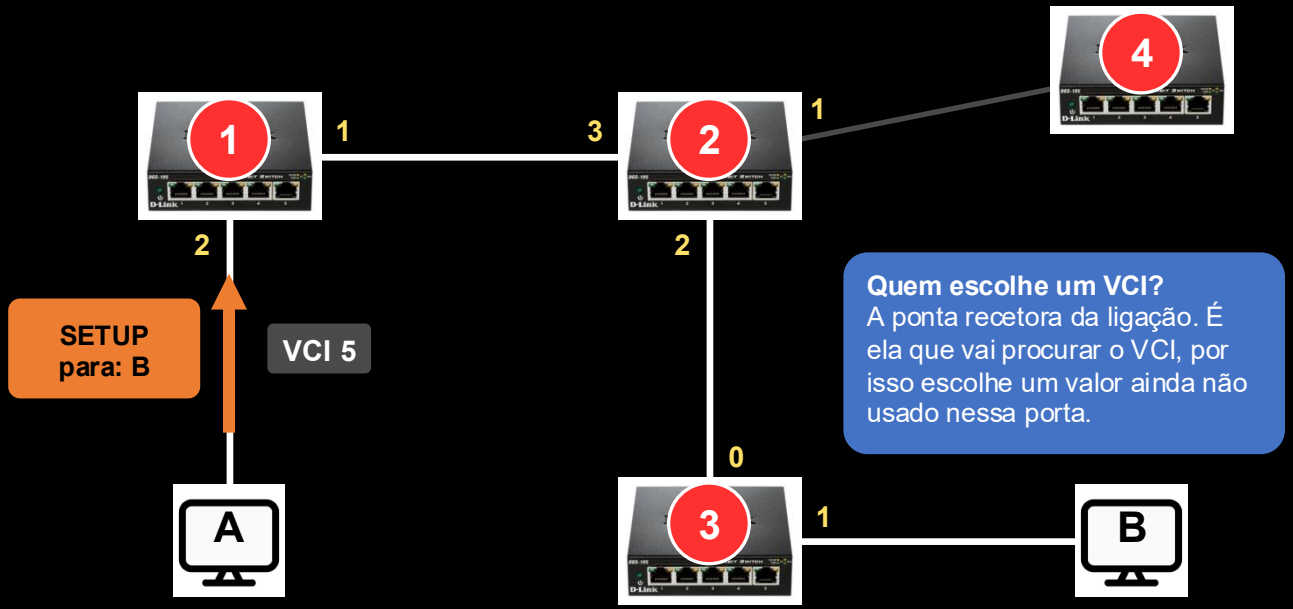
Porta ent.	VCI ent.	Porta saída	VCI saída

Tabela VC, switch 3

Porta ent.	VCI ent.	Porta saída	VCI saída

O Host A quer uma ligação ao Host B. Primeiro, uma mensagem de estabelecimento percorre o caminho A → 1 → 2 → 3 → B. Transporta o endereço completo de B — a única vez que esse endereço viaja — para que cada switch possa escolher a sua porta de saída.

O Pedido Chega ao Switch 1



Quem escolhe um VCI?
A ponta recetora da ligação. É ela que vai procurar o VCI, por isso escolhe um valor ainda não usado nessa porta.

Tabela VC, switch 1

Porta ent.	VCI ent.	Porta saída	VCI saída
2	5	1	

Tabela VC, switch 2

Porta ent.	VCI ent.	Porta saída	VCI saída

Tabela VC, switch 3

Porta ent.	VCI ent.	Porta saída	VCI saída

O pedido de estabelecimento chega pela porta 2. O switch 1 escolhe o VCI 5 para o tráfego vindo de A, e usa o endereço de B para escolher a porta de saída 1. O VCI de saída fica em branco: é o switch seguinte que o escolhe.

O Pedido Chega ao Switch 2

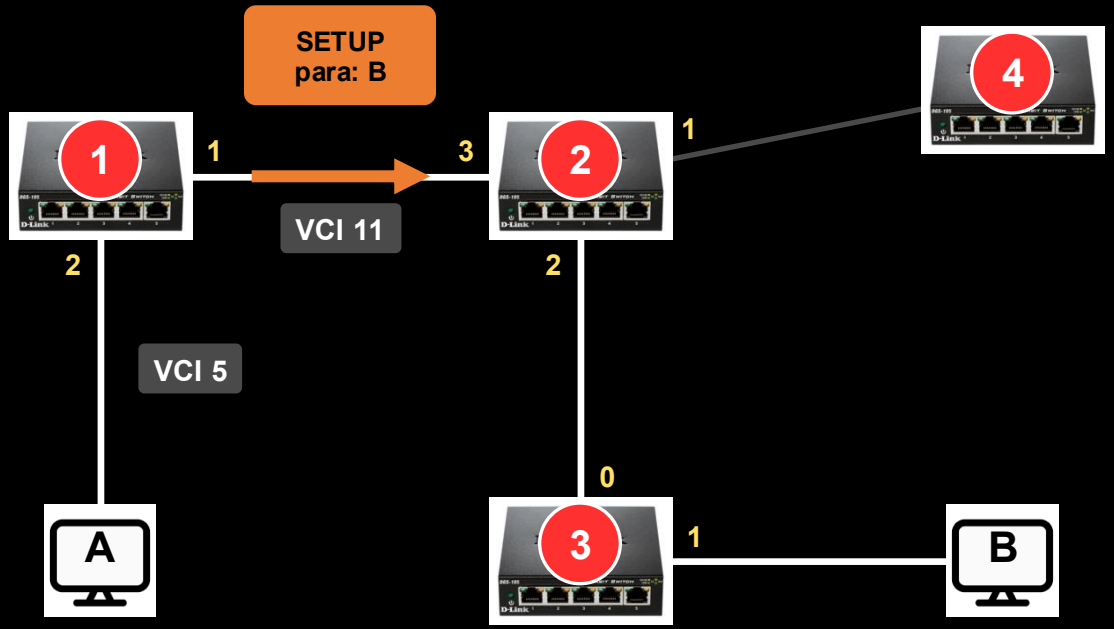


Tabela VC, switch 1

Porta ent.	VCI ent.	Porta saída	VCI saída
2	5	1	

Tabela VC, switch 2

Porta ent.	VCI ent.	Porta saída	VCI saída
3	11	2	

Tabela VC, switch 3

Porta ent.	VCI ent.	Porta saída	VCI saída

O switch 1 encaminha exatamente o mesmo pedido pela porta 1. O switch 2 faz o mesmo trabalho: porta de entrada 3, escolhe o VCI 11, porta de saída 2 em direção a B. O switch 1 ainda não sabe do 11.

O Pedido Chega ao Switch 3

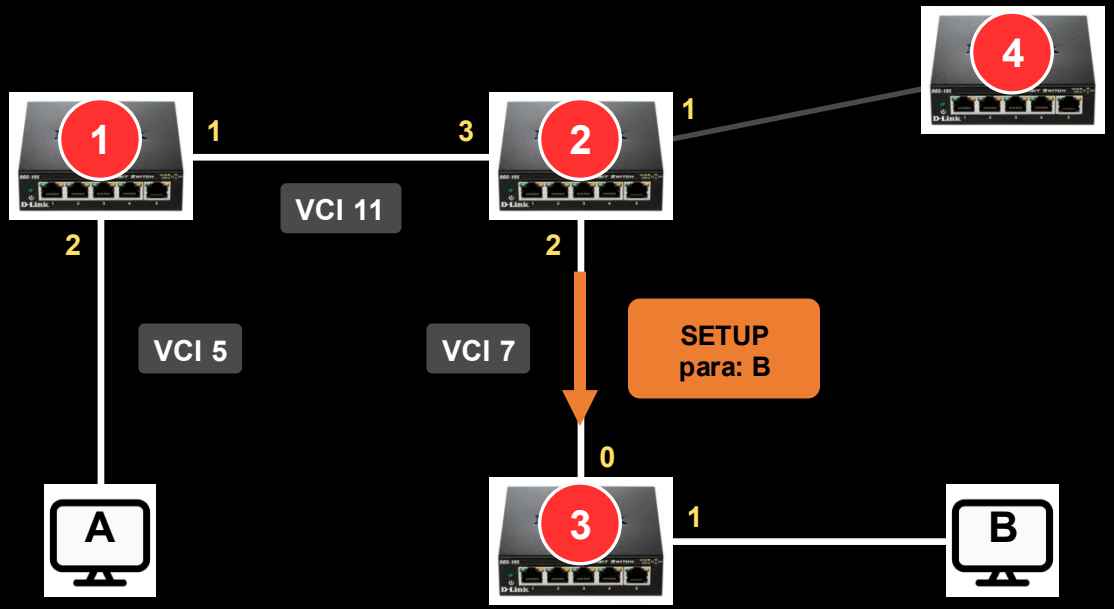


Tabela VC, switch 1

Porta ent.	VCI ent.	Porta saída	VCI saída
2	5	1	

Tabela VC, switch 2

Porta ent.	VCI ent.	Porta saída	VCI saída
3	11	2	

Tabela VC, switch 3

Porta ent.	VCI ent.	Porta saída	VCI saída
0	**7**	**1**	

Switch 3: porta de entrada 0, escolhe o VCI 7, porta de saída 1. O seu VCI de saída é escolha de quem estiver na outra ponta da porta 1 — e esse é o host B.

O Pedido Chega a B

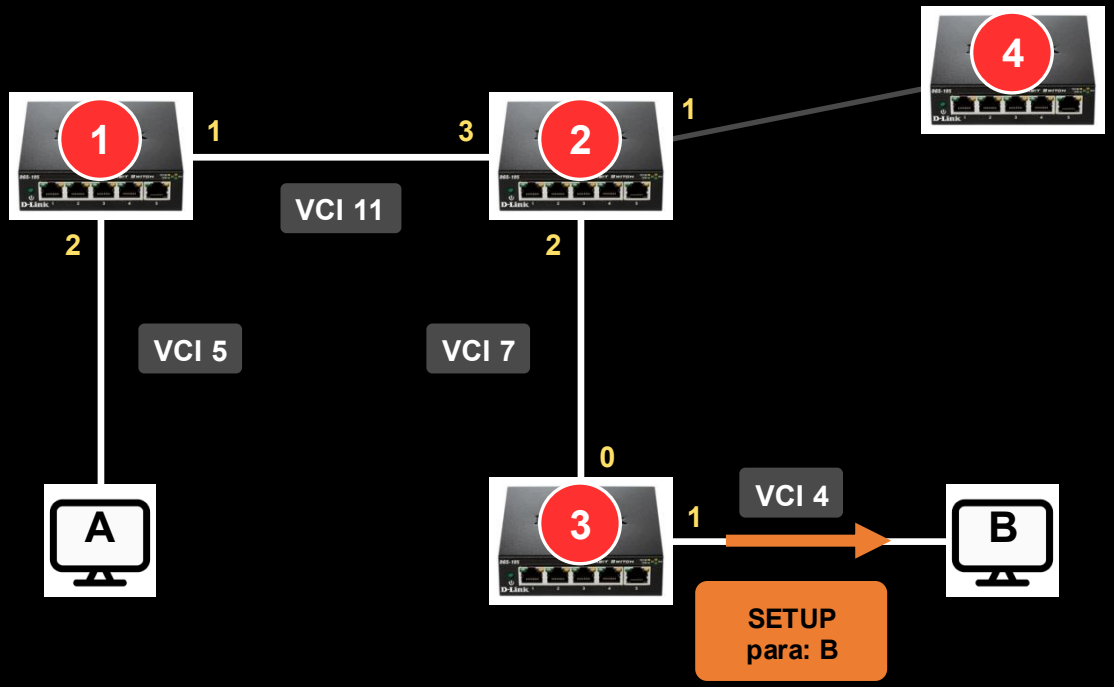


Tabela VC, switch 1

Porta ent.	VCI ent.	Porta saída	VCI saída
2	5	1	

Tabela VC, switch 2

Porta ent.	VCI ent.	Porta saída	VCI saída
3	11	2	

Tabela VC, switch 3

Porta ent.	VCI ent.	Porta saída	VCI saída
0	7	1	

B aceita a ligação e escolhe o VCI 4: todas as tramas desta ligação vão chegar com o 4, e é assim que B a distingue de qualquer outra. Agora cada escolha tem de viajar de volta.

B: “Usem o VCI 4”

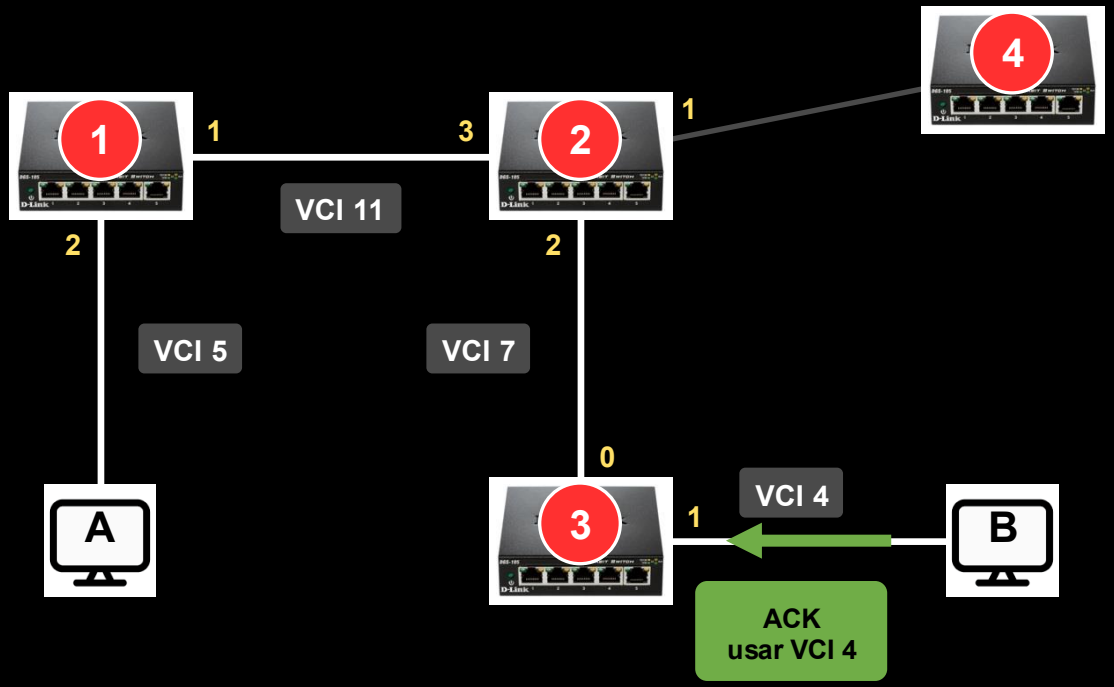


Tabela VC, switch 1

Porta ent.	VCI ent.	Porta saída	VCI saída
2	5	1	

Tabela VC, switch 2

Porta ent.	VCI ent.	Porta saída	VCI saída
3	11	2	

Tabela VC, switch 3

Porta ent.	VCI ent.	Porta saída	VCI saída
0	7	1	4

B responde ao switch 3 com uma confirmação que transporta o VCI que escolheu: 4. Só agora o switch 3 pode completar a sua linha — o último switch fica a saber o seu VCI de saída pela resposta de B.

Switch 3: “Usem o VCI 7”

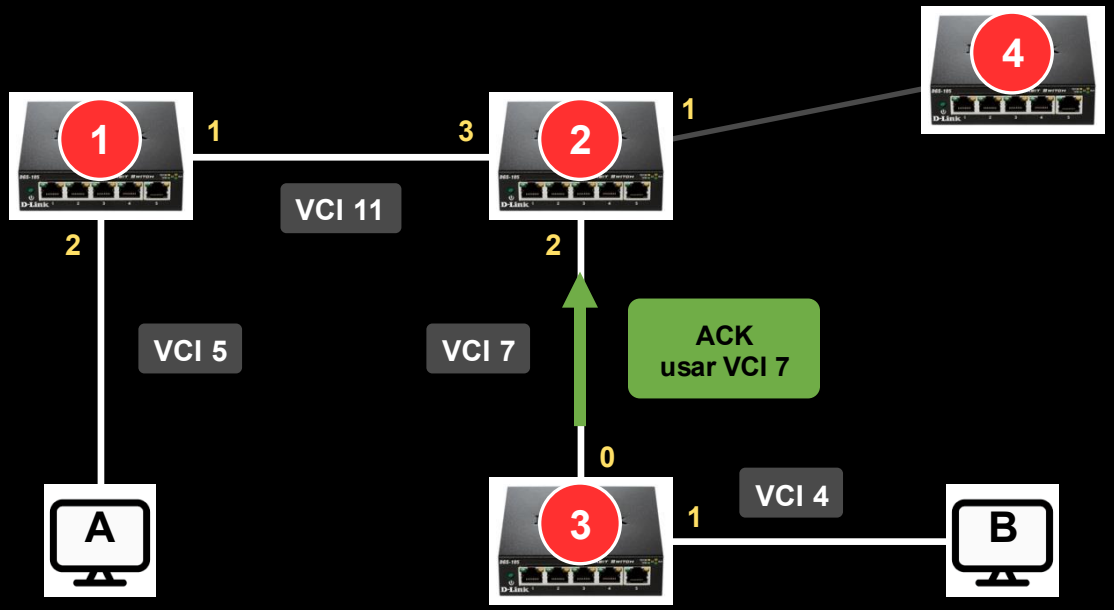


Tabela VC, switch 1

Porta ent.	VCI ent.	Porta saída	VCI saída
2	5	1	

Tabela VC, switch 2

Porta ent.	VCI ent.	Porta saída	VCI saída
3	11	2	7

Tabela VC, switch 3

Porta ent.	VCI ent.	Porta saída	VCI saída
0	7	1	4

O switch 3 passa a sua própria confirmação a montante, com o VCI que escolheu para essa ligação: 7. O switch 2 completa a sua linha.

Switch 2: “Usem o VCI 11”

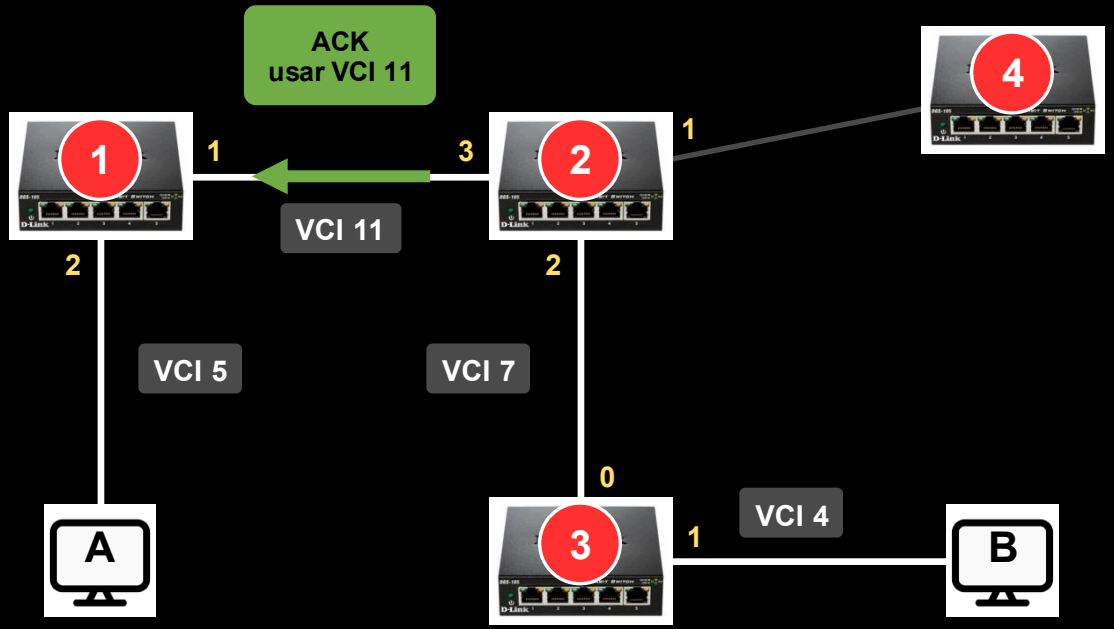


Tabela VC, switch 1

Porta ent.	VCI ent.	Porta saída	VCI saída
2	5	1	11

Tabela VC, switch 2

Porta ent.	VCI ent.	Porta saída	VCI saída
3	11	2	7

Tabela VC, switch 3

Porta ent.	VCI ent.	Porta saída	VCI saída
0	7	1	4

Outra vez o mesmo: o switch 2 diz ao switch 1 para usar o 11. Cada confirmação transporta um valor — o VCI que o seu emissor escolheu para a ligação por onde ela viaja.

Switch 1: “Usem o VCI 5”

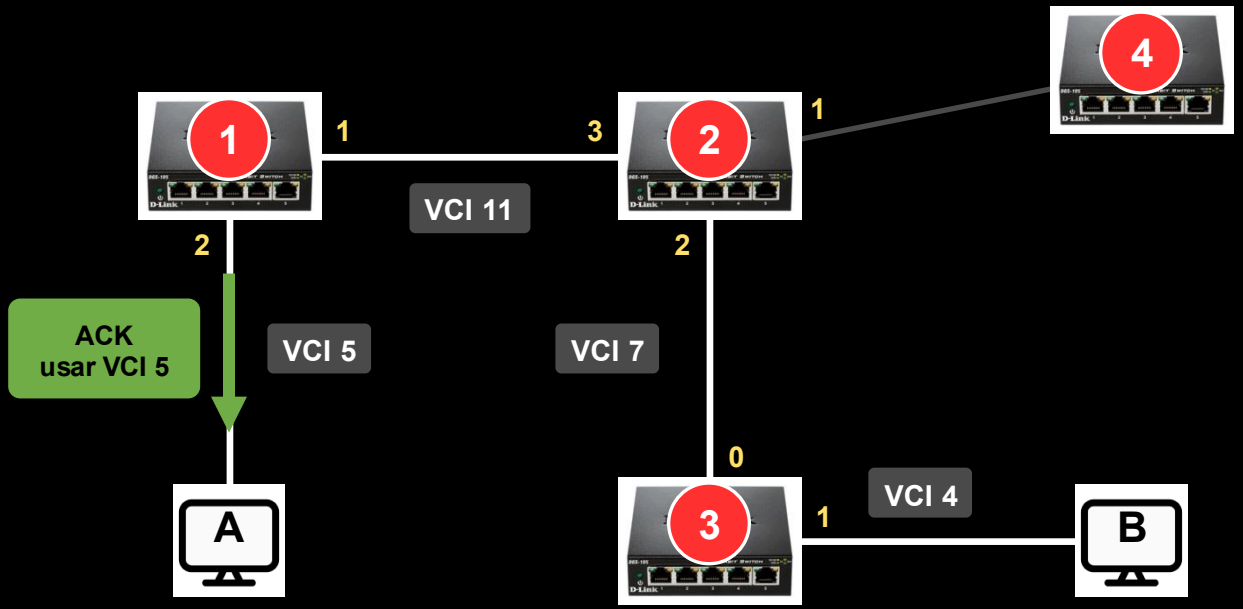


Tabela VC, switch 1

Porta ent.	VCI ent.	Porta saída	VCI saída
2	5	1	11

Tabela VC, switch 2

Porta ent.	VCI ent.	Porta saída	VCI saída
3	11	2	7

Tabela VC, switch 3

Porta ent.	VCI ent.	Porta saída	VCI saída
0	7	1	4

Por fim, o switch 1 diz a A para usar o 5. Todas as tabelas estão completas, e A sabe o que escrever em cada trama. Duas mensagens por salto, uma ida e volta — e a ligação existe.

Dados: o VCI 5 Sai do Host A

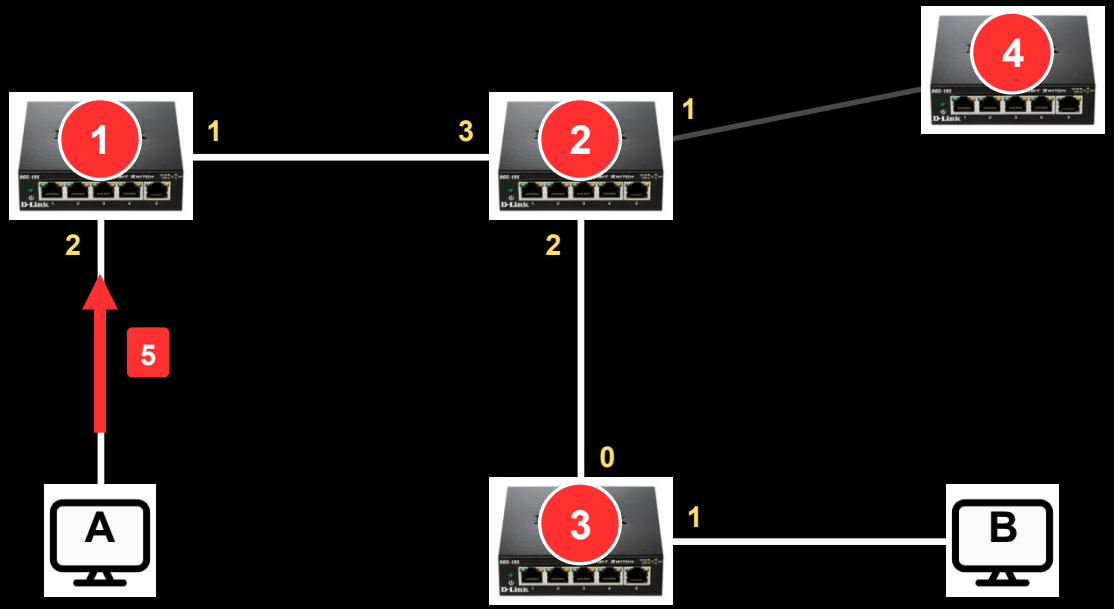


Tabela VC, switch 1

Porta ent.	VCI ent.	Porta saída	VCI saída
2	5	1	11

Tabela VC, switch 2

Porta ent.	VCI ent.	Porta saída	VCI saída
3	11	2	7

Tabela VC, switch 3

Porta ent.	VCI ent.	Porta saída	VCI saída
0	7	1	4

A coloca o VCI 5 no cabeçalho — sem qualquer endereço de destino — e envia a trama para o switch 1.

Procurar, Trocar, Encaminhar

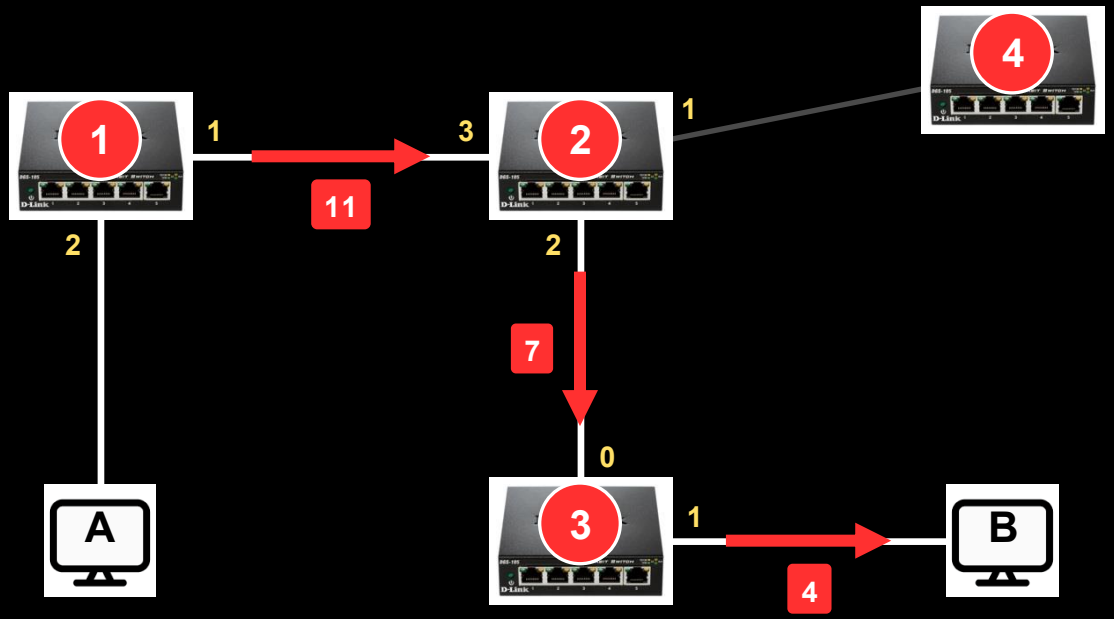


Tabela VC, switch 1

Porta ent.	VCI ent.	Porta saída	VCI saída
2	5	1	11

Tabela VC, switch 2

Porta ent.	VCI ent.	Porta saída	VCI saída
3	11	2	7

Tabela VC, switch 3

Porta ent.	VCI ent.	Porta saída	VCI saída
0	7	1	4

O switch 1 faz corresponder (porta 2, VCI 5), troca para 11 e envia pela porta 1; o switch 2 troca 11 por 7; o switch 3 troca 7 por 4. B recebe o VCI 4 — e sabe que vem de A.

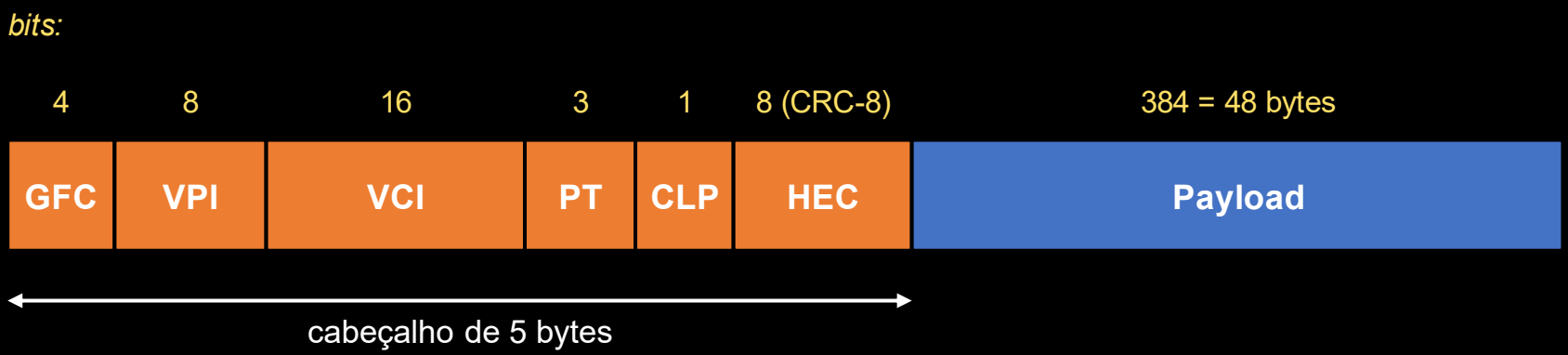
Porquê Circuitos Virtuais?

- **Primeiro o estabelecimento:** nenhuns dados circulam antes de o pedido ir e voltar — pelo menos um RTT, como no handshake do TCP, mas com estado guardado em cada switch ao londo do caminho, não só nos dois hosts.

- **Pesquisas rápidas:** um VCI é pequeno e local, por isso o switch usa-o diretamente como índice da tabela, em vez de procurar um endereço completo.

- **Recursos por ligação:** o estabelecimento pode reservar buffers ou largura de banda em cada salto — ou recusar a ligação antes de enviar dados. A engenharia de tráfego do MPLS assenta nisto.

ATM: Circuitos em Hardware



VPI + VCI = o identificador de circuito virtual que acabaram de ver, trocado em cada salto — uma hierarquia de dois níveis (um caminho que agrupa muitos circuitos), a mesma ideia de agregação dos prefixos CIDR da semana passada.
HEC protege apenas o cabeçalho: um VCI corrompido enviaria a célula para o sítio errado.

Cada célula ATM tem exatamente 53 bytes (o cabeçalho mostrado é o formato usado entre um host e a rede: a UNI). O tamanho fixo tornou mais fácil construir switches de hardware paralelos e rápidos, e evitou que uma célula de voz ficasse à espera atrás de um pacote de dados enorme.

O ATM Hoje

- **O seu auge:** nos anos 90, o ATM era a escolha das operadoras telefónicas para backbones de alta velocidade, e levou a primeira geração de banda larga ADSL até às casas.

- **O que aconteceu:** a Ethernet foi ficando mais rápida e mais barata, e o IP ganhou como camada comum; a partir dos anos 2000, as operadoras substituíram o ATM por Ethernet e IP/MPLS.

- **Hoje:** legado — sem novas instalações; sobrevive em equipamento mais antigo e em algumas linhas ADSL que ainda perduram. A sua ideia central, a troca de etiquetas, continua viva no MPLS.

Circuitos Virtuais Hoje: MPLS

- **MPLS (Multiprotocol Label Switching):** normalizado por volta de 2001, corre dentro das redes dos operadores, por baixo do IP.

- **O mesmo truque que um VCI:** o primeiro router junta uma etiqueta curta a cada pacote; cada router interior procura-a, troca-a e encaminha — por um caminho estabelecido antes.

- **Porque é que os operadores o usam:** para conduzir o tráfego por caminhos à sua escolha, e para construir redes privadas (VPNs) para empresas sobre um backbone partilhado. Os hosts nunca o veem.

Falha de Ligação: Circuitos

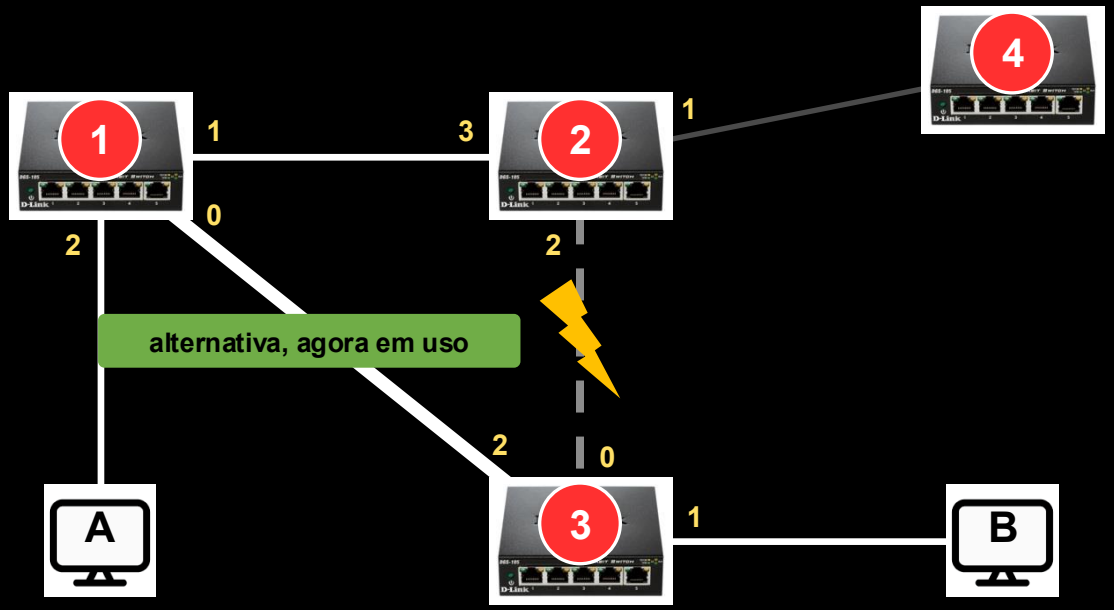


Tabela VC, switch 1

Porta ent.	VCI ent.	Porta saída	VCI saída
2	5	1	11

Tabela VC, switch 2

Porta ent.	VCI ent.	Porta saída	VCI saída
3	11	2	7

Tabela VC, switch 3

Porta ent.	VCI ent.	Porta saída	VCI saída
0	7	1	4

Todas descrevem o caminho antigo.

A ligação 2–3 falha; a ligação alternativa 1–3 passa a transportar o tráfego. Todas as linhas ainda descrevem o caminho antigo pelo switch 2 (switch 1: porta de saída 1; switch 2: porta de saída 2; switch 3: porta de entrada 0), por isso o circuito tem de ser desfeito e estabelecido de novo ao longo de 1–3.

Pacotes vs. Circuitos Virtuais

Conclusão:
A comutação de pacotes (o que faz cada switch Ethernet que viram hoje) paga uma pesquisa de endereço em cada trama, mas não precisa de estabelecimento e lida bem com falhas. Os circuitos virtuais pagam um estabelecimento único e estado por ligação em cada switch, em troca de uma etiqueta curta em cada trama de dados. Ambos continuam em uso — os circuitos virtuais vivem no MPLS.

Haverá uma Terceira Alternativa?

Pergunta:
A comutação de pacotes precisa de um endereço de destino completo em cada trama. Os circuitos virtuais trocam isso por uma etiqueta curta por ligação, à custa de uma fase de estabelecimento. Haverá forma de evitar as duas coisas — sem endereço completo, sem estabelecimento — colocando a informação de encaminhamento noutro sítio?

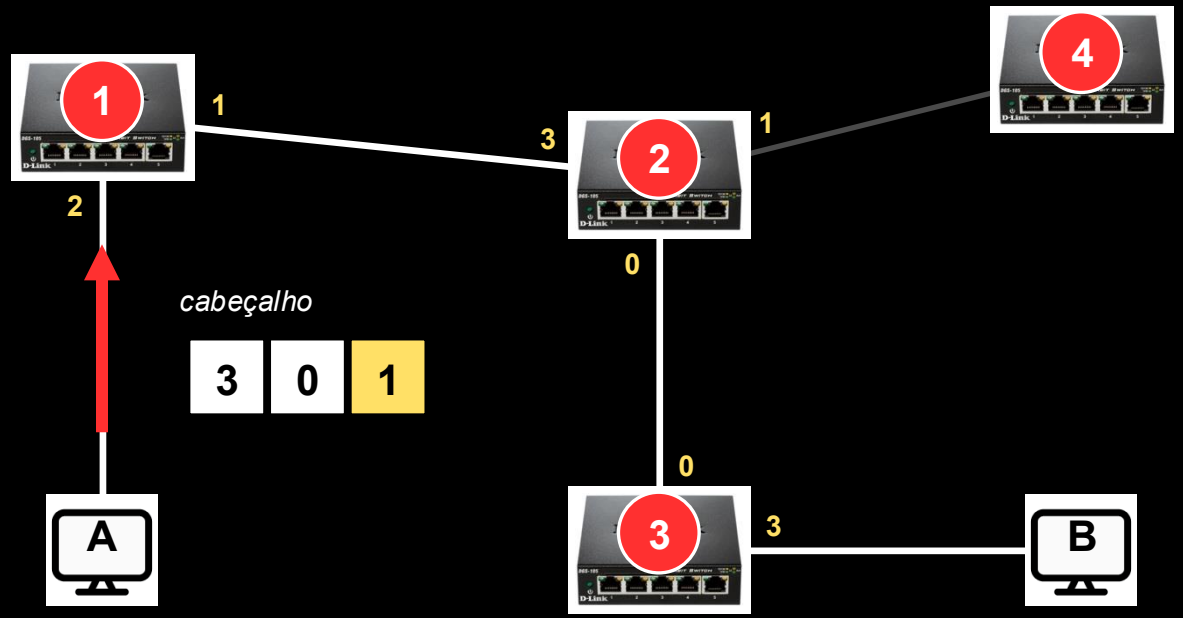
Encaminhamento na Origem

- **A ideia:** a origem calcula antecipadamente o caminho inteiro e escreve-o no cabeçalho do pacote — uma porta de saída por switch (na prática, muitas vezes é o primeiro switch que o escreve).

- **O trabalho de cada switch quase desaparece:** ler uma entrada do cabeçalho, enviar o pacote por essa porta, e passar adiante o resto da lista — sem tabela, sem pesquisa, sem estabelecimento.

- **O custo muda de sítio:** a origem tem de conhecer toda a topologia para construir um caminho correto, e o cabeçalho tem de levar uma entrada por salto.

A Sair do Host A

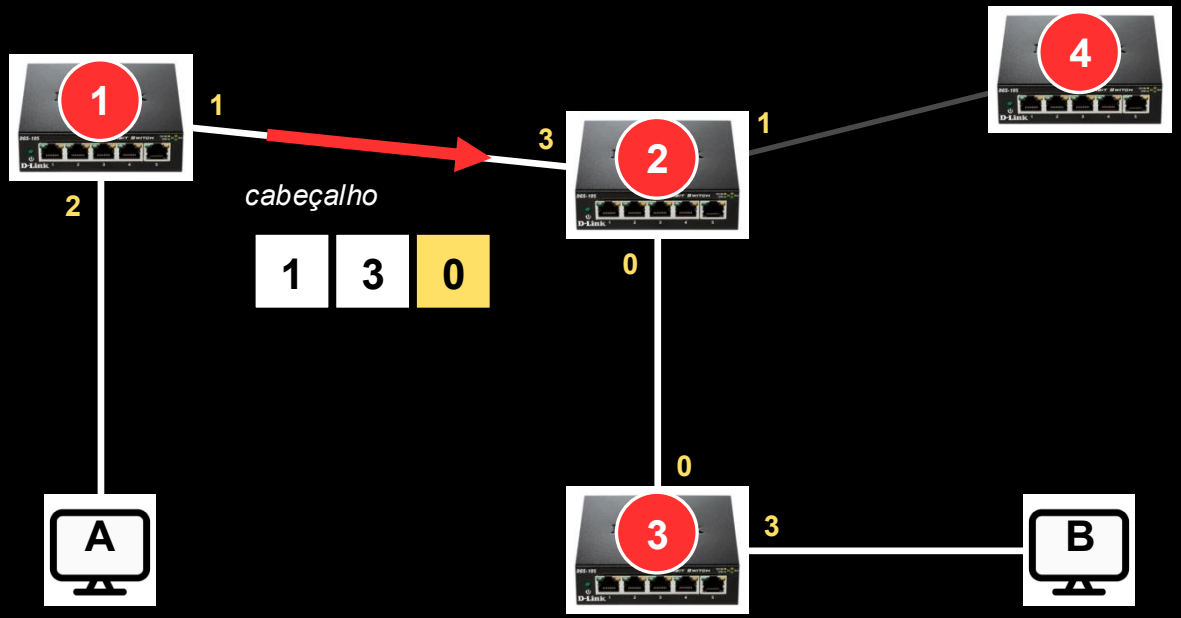


O switch 1 lê 1:
sai pela porta 1.

Depois roda o cabeçalho.

A escreve o caminho inteiro no cabeçalho: (3, 0, 1). Cada switch lê a entrada mais à direita.

Depois do Switch 1

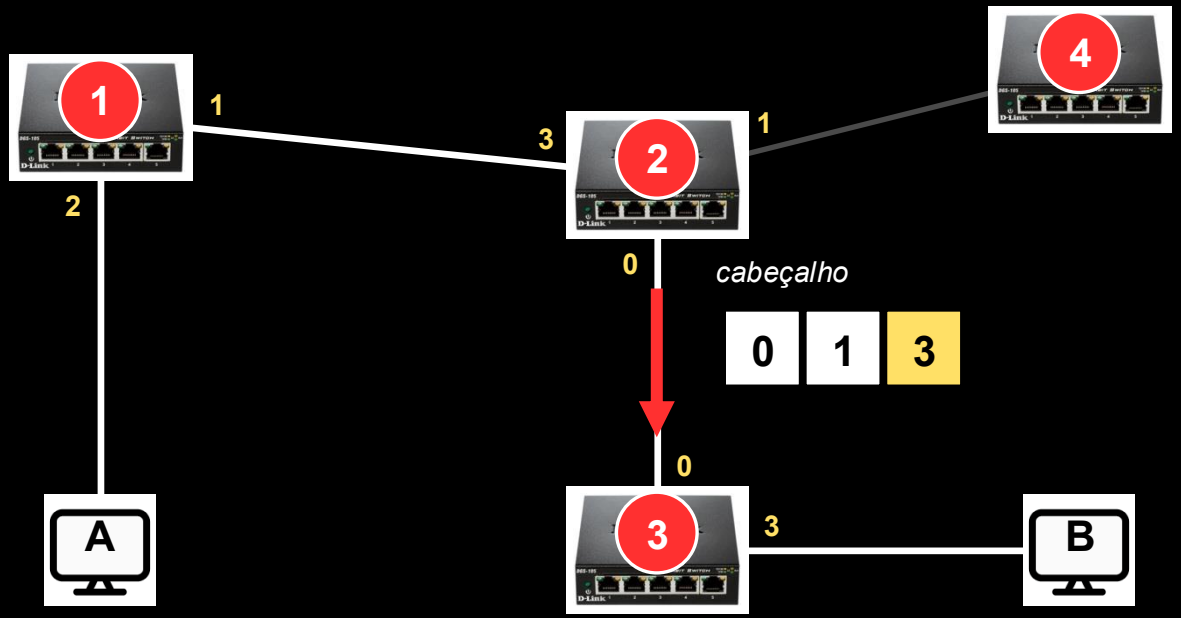


O switch 2 lê 0:
sai pela porta 0.

Sem tabela, sem pesquisa — só ler e rodar.

O switch 1 enviou o pacote pela porta 1 e rodou o cabeçalho para (1, 3, 0). O switch 2 lê 0.

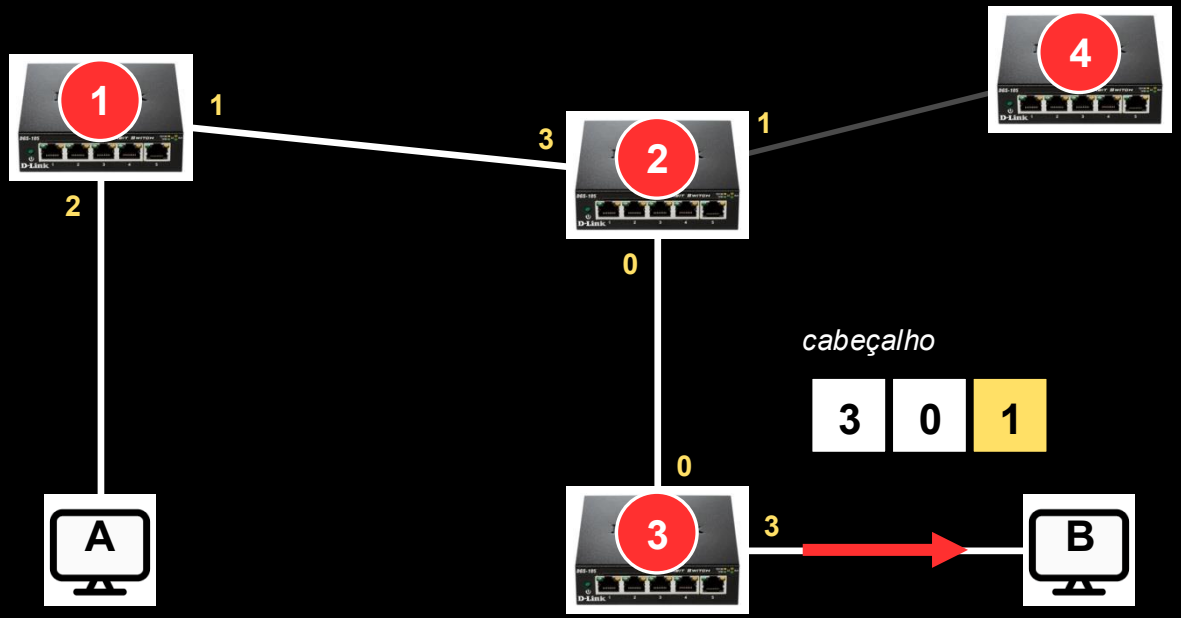
Depois do Switch 2



O switch 3 lê 3:
sai pela porta 3.

O cabeçalho é agora (0, 1, 3). O switch 3 lê 3 e envia o pacote pela porta 3, diretamente para B.

Entregue, Caminho Intacto



A rotação não custa nada a mais e entrega a B uma cópia completa do caminho.

Mas A teve de conhecer a topologia para o construir.

Depois de três rotações, B recebe outra vez (3, 0, 1): um registo completo do caminho que o pacote fez, que pode ajudar B a descobrir o caminho de volta.

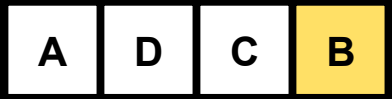
O Cabeçalho: Três Formas

Rotação

a entrar no switch



a sair do switch



entrada usada passa à frente

Remoção

a entrar no switch



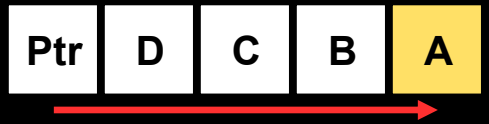
a sair do switch



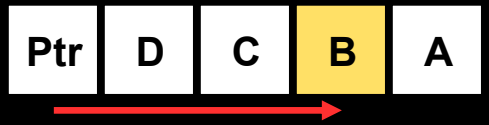
a entrada usada é removida

Ponteiro

a entrar no switch



a sair do switch

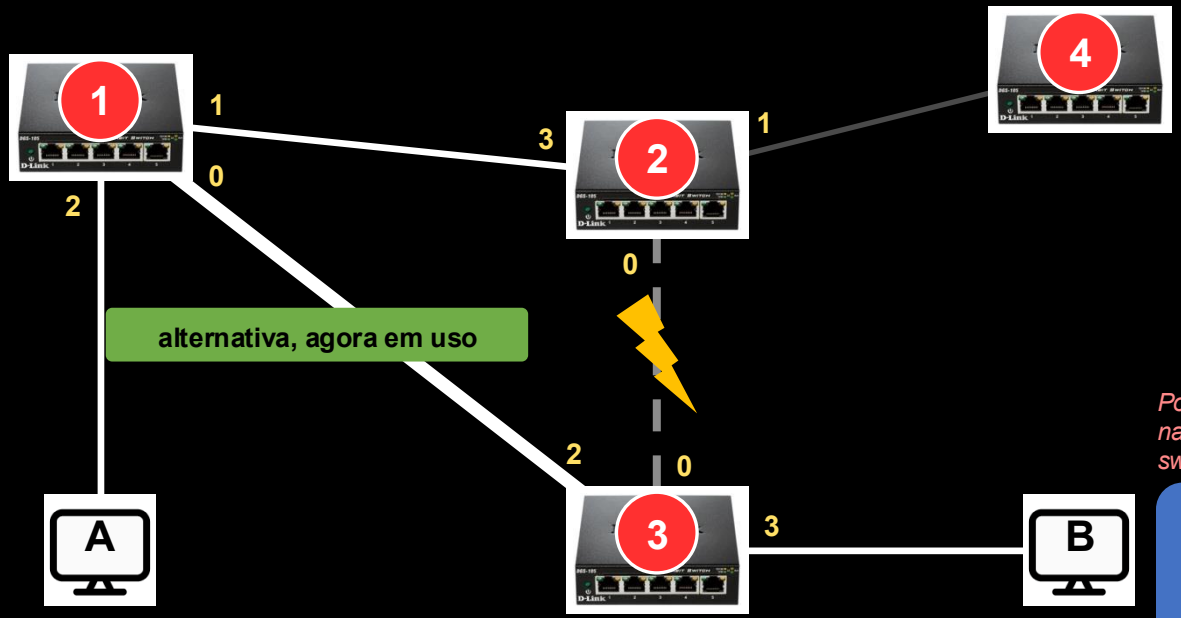


só o ponteiro se move

Os rótulos leem-se da direita para a esquerda.

Cada switch precisa da entrada com o rótulo A; o switch seguinte precisa de B. A rotação mantém tudo, a remoção encolhe o cabeçalho, um ponteiro deixa-o intacto.

Falha de Ligação: Caminhos



o caminho antigo de A

~~3~~ ~~0~~ ~~1~~

o novo caminho de A

3 0

Portas como no exemplo de encaminhamento na origem: a ligação falhada é a porta 0 do switch 2.

Cada porta gravada num caminho é uma promessa sobre a topologia — e a topologia acabou de a quebrar.

O caminho guardado por A, (3, 0, 1), esbarra na ligação morta no switch 2, e nenhum switch o pode corrigir: A tem de conhecer a nova topologia e escrever um novo caminho. Um switch de datagramas recupera sozinho: depois de limpas as entradas obsoletas, inunda e reaprende.

Encaminhamento na Origem Hoje

- **Na Ethernet comutada:** praticamente ausente — os switches reais usam o modelo de comutação de pacotes e a auto-aprendizagem que viram hoje.

- **No IP:** há opções de encaminhamento na origem — estrito (todos os saltos) ou solto (alguns pontos de passagem) — mas as redes descartam-nas: permitem contornar firewalls. A versão genérica do IPv6 (routing header de Tipo 0) foi descontinuada em 2007.

- **Onde a ideia regressou:** Segment Routing, em redes de operadores e centros de dados — em geral é o router de entrada, não o host, que escreve a lista de saltos (etiquetas MPLS ou IPv6).

Três Paradigmas, Lado a Lado

Conclusão:
A comutação de pacotes põe um endereço completo em cada trama e cada switch decide sozinho, sem estabelecimento. Os circuitos virtuais passam esse custo para um estabelecimento único, em troca de uma etiqueta curta em cada trama. O encaminhamento na origem volta a mudar o custo de sítio — para o emissor, que calcula o caminho inteiro à partida e que torna a operação dos switches muito mais simples. Três respostas diferentes à mesma pergunta: quem decide para onde vai este pacote, e quando?

Resumo

- **Auto-aprendizagem:** um switch constrói a sua própria tabela de encaminhamento de graça, apenas observando o endereço de origem de cada trama que vê.

- **O problema dos ciclos:** as ligações redundantes são boas para a fiabilidade, mas combinadas com a inundação e sem memória de “já vi isto”, criam uma tempestade de broadcast infinita.

- **Spanning Tree Protocol:** os switches elegem uma raiz, calculam distâncias, e bloqueiam todos os caminhos redundantes menos um — mantendo a ligação de reserva ligada, apenas sem encaminhar, até ser precisa.

- **Três paradigmas de comutação:** a comutação de pacotes procura um endereço completo de cada vez, os circuitos virtuais trocam isso por um estabelecimento único e uma etiqueta curta, o encaminhamento na origem passa a decisão para o emissor.

O Que Vem a Seguir

Passagem de testemunho:
cada switch aqui decidiu para onde encaminhar usando um endereço de destino que já estava na trama. Próxima aula: o que é realmente esse endereço, e o formato real da trama onde ele existe — Formato da trama Ethernet, entre outras coisas.

Bibliografia

- Peterson, L. L., & Davie, B. S. (2019). Computer Networks: A Systems Approach (Release Version 6.1). Capítulo 3, Secções 3.1 (Switching Basics) e 3.2 (Switched Ethernet); Secção 4.4 (Multiprotocol Label Switching).
- Perlman, R. (1985). An Algorithm for Distributed Computation of a Spanning Tree in an Extended LAN. ACM SIGCOMM Computer Communication Review, 15(4).
- IEEE (2004). IEEE 802.1D-2004: Media Access Control (MAC) Bridges (Rapid Spanning Tree; substituiu o STP original, normalizado pela primeira vez no IEEE 802.1D-1990).

Bibliografia (cont.)

- ITU-T (1999). Recommendation I.361: B-ISDN ATM Layer Specification.
- Rosen, E., Viswanathan, A., & Callon, R. (2001). Multiprotocol Label Switching Architecture. RFC 3031.
- Filsfils, C., Previdi, S., Ginsberg, L., Decraene, B., Litkowski, S., & Shakir, R. (2018). Segment Routing Architecture. RFC 8402.
- Abley, J., Savola, P., & Neville-Neil, G. (2007). Deprecation of Type 0 Routing Headers in IPv6. RFC 5095.

©2026 NOVA FCT